

**ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA**
Universidad de Córdoba



TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Diseño Modular e Incremental para la Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste

Autor

Felipe López Castro

Directores

Joaquín Cuéllar Padilla
Dr. Fernando León García

Curso 2025-2026



Agradecimientos

En primer lugar, quiero expresar mi más profundo agradecimiento a mis padres, que han estado a mi lado todo el tiempo y han sido mi principal fuente de apoyo y motivación. Han celebrado todos mis logros y me han animado en mis peores momentos. Sin su ayuda no hubiera llegado hasta aquí.

Igualmente, quiero agradecer a mis amigos del pueblo, que por mucho tiempo que pasemos sin vernos, siempre acabamos reuniéndonos para recordar lo que significa disfrutar de la vida. Con ellos he pasado muchos de los mejores momentos de mi vida y así seguirá siendo por siempre.

No puedo olvidar a mis compañeros de clase, quienes se han convertido en grandes amigos. Han sido personas indispensables en todos estos años de carrera, brindando su ayuda y su apoyo en los peores momentos. Gracias a ellos me llevo grandes recuerdos de esta etapa de mi vida.

Deseo también darle las gracias a mis profesores de la Escuela Municipal de Música Tenor Pedro Lavirgen, María y Felix. Las clases con ellos han sido los momentos más esperados de la semana, donde me han enseñado valiosas lecciones sobre la música y la vida en general, convirtiéndose para mí en un segundo hogar.

Quiero también agradecer a mis tutores Joaquín y Fernando, por hacer posible la realización de este trabajo. Sus ideas, orientación e infinita paciencia han sido la clave para el correcto desarrollo de este Trabajo Fin de Grado, les estoy profundamente agradecido por todo lo que han hecho por mí.

Por último, quiero darles las gracias al grupo de bajos de la Banda de Cornetas y Tambores de la Coronación de Espinas, por acogerme entre sus filas y convertirse rápidamente en una gran familia. Juntos hemos pasado por innumerables ensayos y grandes conciertos que nunca olvidaré. En especial, quiero agradecer a Virginia, una gran amiga de la banda, quien con su experiencia y consejos me ha ayudado a no rendirme y me ha animado a mejorar en todo momento.

A todos ellos quiero expresar mi más profundo agradecimiento.

Diseño Modular e Incremental para la Navegación Autónoma Contextual en Sistemas Embebidos de Bajo Coste

Resumen

Este Trabajo de Fin de Grado presenta el diseño, implementación y validación de un sistema de navegación autónoma contextual sobre una plataforma robótica de bajo coste (PiCar-B con Raspberry Pi 3 B+). A diferencia de los sistemas reactivos tradicionales basados en infrarrojos, este proyecto desarrolla una arquitectura software modular por capas que integra visión artificial mediante OpenCV y Python.

El vehículo desarrollado es capaz de procesar imágenes en tiempo real para seguir líneas de múltiples colores, adaptar su velocidad según la curvatura y tomar decisiones inteligentes en bifurcaciones mediante la lectura de códigos QR. Los resultados experimentales validan la robustez del sistema en circuitos complejos, demostrando la viabilidad de implementar algoritmos de percepción y control PID avanzados en sistemas embebidos con recursos limitados y bajo coste económico.

Palabras clave: Navegación Autónoma, Visión Artificial, Sistemas Embebidos, Raspberry Pi, OpenCV, Control PID

Modular and Incremental Design for Contextual Autonomous Navigation in Low-Cost Embedded Systems

Abstract

This Final Degree Project presents the design, implementation and validation of a contextual autonomous navigation system on a low-cost robotic platform (PiCar-B with Raspberry Pi 3 B+). Unlike traditional infrared-based reactive systems, this project develops a layered modular software architecture that integrates computer vision using OpenCV and Python.

The developed vehicle is capable of processing images in real time to follow lines of multiple colors, adapt its speed according to curvature and make intelligent decisions at forks by reading QR codes. The experimental results validate the robustness of the system in complex circuits, demonstrating the feasibility of implementing advanced PID perception and control algorithms in embedded systems with limited resources and low economic cost.

Keywords: Autonomous Navigation, Computer Vision, Embedded Systems, Raspberry Pi, OpenCV, PID Control.

Índice general

1. Introducción y contexto	15
1.1. Historia y evolución:	15
1.1.1. Historia y evolución en proyectos educativos:	16
1.2. Estado actual y relevancia:	17
1.3. Enfoque y metodología:	18
2. Objetivos	20
2.1. Objetivo Principal	20
2.2. Objetivos Secundarios	20
3. Antecedentes	21
3.1. Sistemas de Seguimiento Reactivo (Sensores Infrarrojos)	21
3.2. Sistemas de Visión Artificial en Plataformas Educativas	21
3.3. Sistemas de Navegación Industrial (AGV y AMR)	22
3.4. Sistemas de Toma de Decisiones	22
3.5. Análisis Comparativo de Tecnologías de Navegación	22
4. Análisis del problema	24
4.1. Definición del Problema	24
4.1.1. Problema Real	24
4.1.2. Problema Técnico	25
4.2. Identificación de Requisitos	26
4.2.1. Requisitos Funcionales (RF)	26
4.2.2. Requisitos No Funcionales (RNF)	27
4.3. Restricciones	28
4.3.1. Factores Estratégicos	28
4.3.2. Factores de Datos	29
5. Diseño	30
5.1. Diseño Hardware	30
5.1.1. Justificación de las Decisiones de Diseño Hardware	30
Selección del Sensor	30
Selección de la Unidad de Procesamiento:	30
Opción A: Microcontroladores (ESP32 / ESP32-CAM)	31
Opción B: Microcontroladores Estándar (Arduino Uno / Mega)	31
Opción C: Raspberry Pi 3 Model B+ (Seleccionada)	31
Tabla Comparativa	32
Selección del Lenguaje: Python vs. C++	32

5.1.2.	Especificaciones Hardware	32
	Plataforma de Procesamiento: Raspberry Pi	32
5.1.3.	Sistema de Actuadores y Sensores	33
5.1.4.	Componentes del Hardware	34
5.1.5.	Diagrama de conexiones del robot	36
5.2.	Diseño Software	38
5.2.1.	Modelo de Datos	38
	Arquitectura por capas	38
	Diagrama de Clases	39
	Capa 2: Abstracción de Hardware (HAL)	39
	Capa 3: Visión y Percepción	41
	Capa 4: Lógica de control	43
	Capa 5: Aplicación	44
5.2.2.	Modelo Procedimental	59
5.2.3.	Flujo de visión	61
5.2.4.	Flujo de Control	63
	Conexión Cliente-Servidor	65
6.	Pruebas	68
6.1.	Prueba 1: Conducción Autónoma con Sensor Infrarrojo	68
6.1.1.	Descripción de los Componentes Utilizados	68
6.1.2.	Funcionamiento del Sistema	69
6.1.3.	Circuito Desarrollado	70
6.1.4.	Requisitos a Cumplir	70
6.1.5.	Resultados Obtenidos	71
6.1.6.	Conclusiones de la Prueba 1	71
6.2.	Prueba 2: Conducción Autónoma con Visión Artificial	72
6.2.1.	Descripción de los Componentes Utilizados	72
6.2.2.	Funcionamiento del Sistema	72
6.2.3.	Circuito Desarrollado	73
6.2.4.	Requisitos a Cumplir	74
6.2.5.	Resultados Obtenidos	75
6.2.6.	Conclusiones de la Prueba 2	75
6.3.	Prueba 3: Conducción Autónoma Completa	77
6.3.1.	Descripción de los Componentes Utilizados	77
6.3.2.	Funcionamiento del Sistema	77
6.3.3.	Circuito Desarrollado	78
6.3.4.	Requisitos a Cumplir	79
6.3.5.	Resultados Obtenidos	80
6.3.6.	Conclusiones de la Prueba 3	80
6.4.	Prueba 4: Prueba del Módulo TrackLine	82
6.4.1.	Descripción de los Componentes y el Sistema	82
6.4.2.	Circuito Desarrollado	82
6.4.3.	Requisitos a Cumplir	82
6.4.4.	Resultados Obtenidos	82
6.4.5.	Conclusiones de la Prueba 4	84
6.5.	Prueba 5: Prueba de Búsqueda de línea	85
6.5.1.	Descripción de los Componentes y el Sistema	85

6.5.2.	Circuito Desarrollado	85
6.5.3.	Requisitos a Cumplir	86
6.5.4.	Resultados Obtenidos	86
6.5.5.	Conclusiones	86
7.	Conclusiones	87
7.1.	Conclusiones Generales	87
7.2.	Grado de Cumplimiento de los Objetivos	87
7.2.1.	Validación de Requisitos Funcionales	87
7.2.2.	Validación de Requisitos No Funcionales	89
7.2.3.	Conclusión sobre los Objetivos	89
7.3.	Retos Técnicos y Limitaciones	90
7.4.	Líneas de Trabajo Futuro	90
A.	Manual de Usuario	96
A.1.	Requisitos Previos	96
A.2.	Montaje e instalación del robot	96
A.3.	Instalación del Entorno Software	97
A.3.1.	1. Preparación del Sistema Operativo	97
A.4.	Ejecución del servidor y el cliente	98

Índice de figuras

1.	Esquema de conexiones.	34
2.	Diagrama de conexiones del robot.	36
3.	Diagrama de conexión de la cámara.	37
4.	Diagrama de la Estructura por Capas del Sistema.	39
5.	Secuencia de inicialización del bus I2C y servos.	40
6.	Diagrama de clases de la capa de Visión y Percepción.	43
7.	Diagrama de las Clases del Sistema.	46
8.	Diagrama de flujo general del robot.	60
9.	Diagrama de flujo de la visión del robot.	62
10.	Diagrama de flujo de la lógica de control del robot.	64
11.	Diagrama de secuencia: Protocolo de conexión y comunicación entre el cliente y el servidor.	67
12.	Vista del robot.	69
13.	Vista del sensor infrarrojo.	69
14.	Vista del Primer Circuito.	70
15.	Segunda Versión del Robot	72
16.	Vista del Segundo Circuito.	74
17.	Tercera Versión del Robot	77
18.	Vista del Tercer Circuito.	79
19.	Activación del módulo <code>trackLine</code> desde la interfaz del cliente	83
20.	Desactivación del módulo <code>trackLine</code> desde la interfaz del cliente	83
21.	Vista del robot buscando la línea	85
22.	Vista final del robot ensamblado.	97
23.	Interfaz del cliente	99

Índice de tablas

1.	Comparativa entre tecnologías de guiado: IR, Visión y AGV.	23
2.	Requisitos funcionales (RF) del sistema.	27
3.	Requisitos no funcionales (RNF) del sistema.	28
4.	Comparativa técnica de plataformas de control.	32
5.	Componentes hardware del robot.	35
6.	Asignación de pines GPIO para los LEDs frontales.	41
7.	Resumen de las clases representadas en el diagrama UML de arquitectura.	47
8.	Resumen de las clases restantes representadas en el diagrama UML de arquitectura.	48
9.	Relación de atributos de la clase Functions	49
10.	Relación de atributos de la clase Functions (continuación).	50
11.	Descripción de los métodos de la clase Functions	51
12.	Descripción de los atributos y métodos de la clase RobotLight	52
13.	Descripción de los atributos y métodos de la clase CameraEvent	53
14.	Descripción de los atributos y métodos de la clase abstracta BaseCamera	54
15.	Descripción de los atributos de la clase Camera	55
16.	Descripción de los métodos de la clase Camera	56
17.	Descripción de los atributos de la clase CVProcessor	57
18.	Descripción de los métodos de la clase CVProcessor	58
19.	Requisitos funcionales (RF) de la Primera Prueba.	70
20.	Requisitos funcionales (RF) de la Segunda Prueba.	75
21.	Requisitos funcionales (RF) de la Tercera Prueba.	80
22.	Grado de cumplimiento de los Requisitos Funcionales (RF).	88
23.	Grado de cumplimiento de los Requisitos No Funcionales (RNF).	89

Capítulo 1

Introducción y contexto

Los sistemas embebidos [1] han adquirido un papel central en el desarrollo de tecnologías modernas debido a su capacidad para integrar procesamiento, comunicación y control en soluciones compactas, energéticamente eficientes y de bajo coste. Este auge se ha visto impulsado por la miniaturización de componentes, la aparición de plataformas de desarrollo accesibles como Arduino y Raspberry Pi, además del crecimiento del paradigma del Internet de las Cosas (IoT). Gracias a estas características, los sistemas embebidos constituyen la base tecnológica de numerosas aplicaciones críticas, desde dispositivos médicos hasta vehículos autónomos.

En el ámbito de la movilidad autónoma, los sistemas embebidos permiten la integración directa entre sensores, actuadores y algoritmos de control, posibilitando la navegación en tiempo real. La capacidad de dotar a una máquina de autonomía para desplazarse de un punto A a un punto B sin intervención humana se ha convertido en un pilar esencial para la industria 4.0 y la logística automatizada. Sin embargo, la complejidad de estos sistemas varía enormemente dependiendo de la cantidad de información que el robot es capaz de extraer e interpretar de su entorno.

1.1. Historia y evolución:

Antes de seguir con el apartado, es importante remontarse al principio de la historia de los vehículos sigue-líneas, para entender cómo fue su inicio y evolución con el paso del tiempo,

En torno a 1950, tenemos lo que podríamos considerar el primer caso de una movilidad autónoma primitiva: el caso de “Las tortugas mecánicas de Grey Walter” [2]. El cual consistió en un experimento cuyo objetivo era probar una serie de robots experimentales con forma de tortuga, quienes imitaban la inteligencia viva. Dichos robots eran capaces de explorar su entorno y reaccionar ante él a través de unas celdas fotoeléctricas que hacían de ojos para los robots, convirtiéndose en las primeras máquinas autónomas de la historia.

Poco después, en 1953, la empresa Barrett Electronics introdujo el primer vehículo de guiado automático (AGV) para uso industrial [3]. A diferencia de los modelos pilotados por personas anteriores, este sistema utilizaba un cable situado en el techo, que emitía señales electromagnéticas. El vehículo detectaba estas ondas y las seguía para transportar materiales de forma autónoma en almacenes, marcando el inicio de la logística automati-

zada y demostrando la utilidad práctica de los sistemas de navegación guiada en entornos de trabajo reales, que con el tiempo se ha ido mejorando e implementando en más centros logísticos.

Al inicio de la década de los 60, surgió el ambicioso proyecto del “Stanford Cart” en la Universidad de Stanford [4]. Este hito, que comenzó con un explorador lunar terrestre y uno de los primeros robots con inteligencia artificial, es fundamental para la robótica moderna, ya que fue uno de los primeros intentos de guiar un vehículo utilizando visión artificial mediante una cámara de televisión.

A medida que avanzaba la investigación en la Universidad de Stanford durante la década de los 70, el Stanford Cart que inicialmente era un vehículo operado remotamente, el cual estaba limitado a los 2,6 segundos de retardo que ocurrían en la comunicación por radio entre la Tierra y la Luna, evolucionó a un sistema verdaderamente autónomo. En esta etapa, el robot logró salir al exterior y seguir una línea blanca a paso humano, enfrentándose a retos que hoy siguen siendo vigentes, como las inconsistencias en la iluminación o las curvas cerradas. Para solventar estas limitaciones, en 1979 se le dotó de visión 3D y una lógica de navegación extremadamente cautelosa: el vehículo avanzaba un metro y se detenía durante casi 15 minutos para procesar su entorno y recalcular su ruta. Aunque tardó cinco horas en recorrer apenas 20 metros entre obstáculos, este experimento marcó el nacimiento de la navegación autónoma sin intervención humana, un hito que hoy se conserva como pieza histórica en el Computer History Museum [5].

En 1984, Valentino Braitenberg propuso una base teórica revolucionaria con sus “Vehículos de Braitenberg”, Esta teoría consistía en la construcción de un vehículo que contara con sensores que captaran estímulos y ruedas, equipada cada rueda con su propio motor [6]. Dichos motores se activaban cuando el sensor al que estaban conectados les enviaba una señal. Sus estudios impulsaron el uso de sensores infrarrojos de bajo costo para el seguimiento de líneas, permitiendo que los robots detectaran el contraste entre el blanco y el negro. Esta arquitectura facilitó que la robótica móvil se expandiera fuera de los laboratorios, simplificando la lógica necesaria para mantener al robot sobre su trayectoria.

Finalmente, a partir de 2012, la popularización de placas como la Raspberry Pi [7] y librerías como OpenCV [8], conformada por funciones enfocadas en el procesamiento de imágenes en tiempo real, ha permitido la transición hacia la navegación contextual que vemos hoy. Con estos avances tanto en hardware como en software, hoy en día se pueden llevar a cabo proyectos donde el robot no solo siga la línea, sino que sea capaz de interpretar el color de la línea, además de tomar decisiones lógicas en base a la información captada por el entorno.

1.1.1. Historia y evolución en proyectos educativos:

La exploración de vehículos autónomos a pequeña escala tiene una larga trayectoria en la robótica educativa. Un ejemplo pionero es el Stanford Cart, que ya hemos visto. Asimismo, en 1962 se dio a conocer el robot “Emily” en la revista Popular Electronics, capaz de seguir un recorrido marcado en el piso. Estos primeros prototipos sentaron las bases de los actuales proyectos educativos de vehículos autónomos. En las décadas siguientes, los robots seguidores de línea ganaron popularidad a través de concursos estudiantiles y ferias de ciencia, al punto de ser considerados el “hola mundo” de la robótica móvil educativa [9]. De hecho, la primera competencia internacional Robotrace de vehículos autónomos

velocistas tuvo lugar en 1986 en Japón y dio origen a la categoría simplificada de “Line Follower” que hoy se reconoce en todo el mundo. En años recientes, la democratización de la electrónica con el surgimiento de plataformas de hardware abierto como Arduino y Raspberry Pi, junto con kits didácticos, ha impulsado la construcción de coches autónomos de bajo costo en colegios y universidades [10]. Estos desarrollos han sido documentados extensamente en la literatura, por ejemplo en proyectos basados en robots de bajo coste para enseñanza de control [11] o en iniciativas de aprendizaje activo en robótica móvil.

1.2. Estado actual y relevancia:

Los avances formativos mencionados tienen su reflejo en la industria automotriz, donde la conducción autónoma ha pasado de ser un experimento de laboratorio a una realidad incipiente. Empresas tecnológicas y fabricantes de automóviles invierten intensamente en sistemas autónomos de distintos niveles [12]. Por ejemplo, Waymo (Google/Alphabet) [13] opera desde 2020 servicios comerciales de robotaxis totalmente autónomos en ciudades de Estados Unidos como Phoenix y San Francisco. Sus vehículos han superado ya los 160 millones de kilómetros de conducción autónoma, demostrando una aceleración significativa en el desarrollo de esta tecnología. Por otro lado, Tesla ha equipado todos sus modelos recientes con hardware para conducción autónoma y ofrece un sistema de “Full Self-Driving” [14] en fase beta, el cual permite la conducción automatizada en la mayoría de las condiciones de tráfico aunque todavía requiere la supervisión constante del conductor. De hecho, Tesla inició en 2025 sus primeras pruebas piloto de servicio de taxis autónomos con una pequeña flota de vehículos en Austin, Texas. Los fabricantes tradicionales también han logrado hitos: Mercedes-Benz fue la primera marca autorizada para vender coches de Nivel 3 (automación condicionada) en 2022, incorporando funciones de piloto automático en autopista con su modelo Drive Pilot [15]. Asimismo, Honda introdujo en 2021 un sedán de nivel 3 en el mercado japonés, y otras marcas como BMW, General Motors (Cruise) o Baidu están desarrollando vehículos de nivel 4 (conducción altamente automatizada) en programas piloto [16]. Estos avances actuales indican que la industria se encuentra en una fase de transición: si bien ningún fabricante ofrece aún un Nivel 5 (autonomía total en cualquier escenario) al público general, se han logrado implementaciones concretas de nivel 3 y 4 en entornos controlados, con expectativas de expansión en los próximos años. La conducción autónoma empieza así a perfilarse como una tecnología madura, apoyada por la mejora continua de sensores, algoritmos de aprendizaje automático y potentes unidades de procesamiento en los vehículos modernos [17] [18].

En el contexto de la robótica móvil, la tarea de seguimiento de línea se considera un modelo simplificado pero sumamente ilustrativo de la navegación autónoma. Constituye un análogo directo del mantenimiento de carril que debe efectuar un coche autónomo a escala real; de hecho, muchos algoritmos de control de trayectoria desarrollados en mini-robots seguidores de líneas son extrapolables a sistemas de guiado en carreteras. Además, el seguimiento de rutas predefinidas es común en entornos industriales y logísticos actuales como hemos visto con los vehículos de guiado automático (AGVs) empleados en fábricas y almacenes para transportar materiales eficientemente [19]. Plataformas como los robots de almacén de Amazon –derivadas conceptualmente de los seguidores de línea– han revolucionado la logística interna, optimizando la gestión de inventario y reduciendo tiempos muertos. Hacia el futuro, se espera que los principios de navegación lineal au-

tónoma encuentren aplicación en las smart cities y la movilidad sostenible. Por ejemplo, sistemas de transporte público autónomo (shuttles) podrían operar en carriles dedicados siguiendo rutas establecidas de forma segura y puntual, o pequeños robots de reparto podrían navegar aceras y carriles bici guiándose por marcas en el entorno urbano. Los expertos proyectan que la adopción generalizada de vehículos autónomos conllevará beneficios sociales importantes: mayor seguridad vial al eliminar errores humanos, mejor accesibilidad para personas con movilidad reducida, optimización del tráfico y reducción de emisiones contaminantes gracias a una conducción más eficiente. En conclusión, el sencillo ejercicio académico de hacer seguir una línea a un robot encierra los fundamentos de la navegación autónoma. Su estudio y perfeccionamiento no solo preparan a los ingenieros del mañana, sino que también anticipan soluciones de movilidad inteligente que pronto podrían integrarse en nuestra vida cotidiana, desde almacenes y centros de distribución más automatizados hasta ciudades más seguras, limpias y conectadas.

En la enseñanza de ingeniería informática, electrónica y mecatrónica, los proyectos de vehículos autónomos desempeñan un rol pedagógico fundamental. Construir un robot seguidor de línea u obstáculo supone un desafío integrador que refuerza conocimientos de programación embebida, electrónica analógica/digital y teoría de control [20]. Por ejemplo, la implementación de un sistema de seguimiento de línea exige comprender el procesamiento de señales de sensores (infrarrojos o cámaras) y aplicar algoritmos de control para el accionamiento preciso de motores, mientras que un sistema de detección de obstáculos introduce al estudiante en el uso de sensores ultrasónicos o de proximidad y en la toma de decisiones autónoma. Estas actividades prácticas complementan la formación teórica al brindar experiencia en el diseño, construcción y depuración de sistemas robóticos reales, fortaleciendo habilidades de resolución de problemas y pensamiento sistémico. Según estudios recientes, la inclusión de prácticas con robots autónomos en el ámbito universitario mejora la motivación y el rendimiento de los estudiantes [21]. Además, proyectos más avanzados de visión por computador, como vehículos a escala que reconocen colores o señales para guiarse, exponen al alumnado a técnicas de inteligencia artificial aplicada. En suma, los vehículos autónomos educativos se han consolidado como plataformas de aprendizaje STEM [22] que motivan a los estudiantes y aumentan su confianza e interés en campos tecnológicos de vanguardia.

1.3. Enfoque y metodología:

Este Trabajo Fin de Grado se enmarca en un caso práctico, en el que se ha construido un prototipo de robot sigue-líneas. El enfoque metodológico adoptado es el de un diseño modular e incremental. El desarrollo no busca crear un sistema monolítico desde cero, sino una arquitectura que permita actualizar progresivamente las capacidades del robot. Se parte de la navegación básica para evolucionar hacia la interpretación de estímulos contextuales, validando en cada etapa la robustez del código y la respuesta de los actuadores.

Además de los objetivos técnicos, el proyecto tiene una vocación de divulgación tecnológica. El proyecto combina experimentación práctica con la filosofía del conocimiento abierto, de modo que los resultados obtenidos permitan validar la viabilidad de nuevas estrategias de navegación autónoma a bajo coste y con un enfoque abierto a la comunidad, facilitando que futuros estudiantes puedan iterar sobre esta base.

En conclusión, este Trabajo de Fin de Grado tiene como objetivo general el diseño e implementación de un sistema de navegación autónoma contextual sobre una arquitectura embebida de bajo coste. A continuación, el documento detalla los objetivos, antecedentes, el análisis del problema, el diseño tanto del apartado físico como del digital, y finalmente, las conclusiones a las que se han llegado tras las pruebas del prototipo en entornos reales.

Capítulo 2

Objetivos

El propósito central de este Trabajo Fin de Grado es el diseño, desarrollo e implementación de un sistema de navegación autónoma sobre una plataforma robótica de bajo coste. A diferencia de los sistemas de seguimiento convencionales de carácter puramente reactivo, este proyecto busca dotar al vehículo de una capacidad de interpretación del entorno que le permita adaptar su comportamiento de forma dinámica ante estímulos visuales complejos.

2.1. Objetivo Principal

El objetivo principal de este TFG es diseñar e implementar un sistema embebido de navegación autónoma que, partiendo de un prototipo básico de seguimiento de líneas, integre progresivamente mecanismos de toma de decisiones contextuales. Mediante la combinación estratégica de tecnologías existentes (hardware, software y protocolos), se buscará dotar al sistema de adaptabilidad operativa en entornos dinámicos, evaluando su capacidad para interpretar estímulos del entorno y ajustar su comportamiento en tiempo real.

2.2. Objetivos Secundarios

- Analizar comparativamente tecnologías existentes (placas, sensores, motores, drivers, librerías, etc.) para la construcción de un prototipo de robot sigue-líneas como punto de partida.
- Desarrollar una arquitectura modular que permita actualizar progresivamente las capacidades del robot, desde el seguimiento básico de líneas hasta la interpretación de estímulos contextuales, sin necesidad de rediseñar elementos estructurales.
- Implementar un subsistema de interpretación de estímulos que traduzca variaciones en el entorno (marcas, cambios cromáticos, geometrías, etc.) en comandos operativos para el vehículo (cambios de velocidad, paradas temporizadas, activación de algún actuador, etc.)
- Compartir el conocimiento generado mediante la publicación del desarrollo del proyecto en un repositorio de código de acceso público y correctamente licenciado bajo una licencia de software libre.

Capítulo 3

Antecedentes

Para abordar el diseño de un sistema de navegación autónoma contextual, es fundamental analizar primero el estado del arte en la robótica móvil. En este capítulo se examinan las soluciones tecnológicas predominantes hasta el momento, identificando sus limitaciones para justificar la necesidad de la solución propuesta.

3.1. Sistemas de Seguimiento Reactivo (Sensores Infrarrojos)

Una de las técnicas más difundidas en la robótica móvil educativa es el seguimiento de línea mediante sensores infrarrojos (IR). Tradicionalmente, estos sistemas basan su funcionamiento en componentes de bajo nivel (como el array CNY70). Su lógica es binaria y reactiva, lo que significa que detectan el contraste entre el suelo y la línea, blanco y negro respectivamente, actuando inmediatamente sobre los motores para corregir la trayectoria, buscando mantener el sensor centrado sobre la guía.

Si bien esta solución es eficiente en términos de coste y velocidad de procesamiento, presenta una limitación crítica para el objetivo de este proyecto: la **“ceguera contextual”**.

- **Ausencia de semántica:** Un sensor infrarrojo estándar solo puede determinar la presencia o ausencia de trazo, pero es incapaz de interpretar atributos del entorno. Para estos robots, una línea recta en una zona segura es idéntica a una en zona de peligro.
- **Adaptabilidad nula:** Su capacidad de respuesta frente a escenarios dinámicos o estímulos complejos es reducida [23]. No pueden distinguir colores ni leer señales codificadas, lo que restringe su aplicabilidad a circuitos cerrados y controlados.

3.2. Sistemas de Visión Artificial en Plataformas Educativas

Para superar la barrera de los infrarrojos, la tendencia actual se dirige hacia el uso de cámaras como sensor principal. Esto introduce una nueva dimensión de datos: ya no se

procesa solo una señal de voltaje, sino una matriz de píxeles que contiene información sobre color, geometría y continuidad.

Existen en el mercado kits de desarrollo y proyectos de código abierto (como las implementaciones base para el kit *PiCar-B* [24] o proyectos en *Embedded Rust*) que utilizan visión artificial. Sin embargo, el análisis de estas soluciones revela carencias arquitectónicas comunes:

1. **Procesamiento Monolítico:** La mayoría de los códigos de ejemplo ejecutan la captura de imagen, el procesamiento y el control de motores en un único hilo de ejecución (*single-thread*). Esto provoca que, si el análisis de imagen se retrasa, el control del motor sufra latencia, generando movimientos bruscos.
2. **Visión Reactiva Simple:** Aunque usan cámara, muchos se limitan a replicar el comportamiento de los sensores IR: detectar el centro de una línea de un color fijo y aplicar un PID. Rara vez implementan lógica de negocio basada en múltiples perfiles de color o detección de patrones más complejos.

3.3. Sistemas de Navegación Industrial (AGV y AMR)

En el extremo opuesto del espectro tecnológico se encuentran los Vehículos de Guiado Automático (AGV) utilizados en la industria 4.0, que como hemos visto anteriormente son descendientes de los sigue-líneas. Estos sistemas utilizan fusión de sensores (LIDAR, cámaras estereoscópicas) y navegación SLAM [25]. Aunque representan el ideal de autonomía, su complejidad y coste de hardware los hacen inviables para el ámbito de los sistemas embebidos de bajo coste y propósito educativo que ocupa este TFG.

3.4. Sistemas de Toma de Decisiones

En cuanto a la toma de decisiones en bifurcaciones o cruces, existen diversos enfoques según la información disponible: algunos robots siguen señales pintadas en el suelo (como flechas o marcas especiales) para decidir el rumbo, mientras que otros se basan en patrones en la línea (por ejemplo, una interrupción o variación en la línea) o incluso en instrucciones preprogramadas. La implementación de estas capacidades suele requerir algoritmos de mayor complejidad, que combinan la información de múltiples sensores.

3.5. Análisis Comparativo de Tecnologías de Navegación

A continuación, se presenta una comparativa técnica entre los robots seguidores de línea tradicionales (basados en infrarrojos), los sistemas basados en visión artificial y los Vehículos de Guiado Automático (AGV) de uso industrial.

Característica	Robot IR (Básico)	Robot Visión (Cámara)	AGV Industrial
Principio de Funcionamiento	Detección de contraste (Reflectancia)	Procesamiento de imagen matricial	Fusión de sensores (Lidar/Magnético/-Visión)
Ancho de Banda de Datos	Muy bajo (1 bit por sensor)	Alto (Matrices RGB)	Masivo (Nube de puntos + Vídeo)
Capacidad Contextual	Nula (Ciego al entorno)	Media (Detección de formas/colores)	Alta (Mapeado y localización)
Flexibilidad de Ruta	Rígida (Física)	Programable (Lógica)	Dinámica (Navegación libre)
Tolerancia a Fallos	Baja (Se pierde si sale de la línea)	Media (Recuperación por algoritmo)	Alta (Redundancia de seguridad)
Interacción con Señales	No (Solo sigue el trazo)	Posible (QR, Color, Formas)	Nativa (RFID, DataMatrix, Wifi)
Hardware de Control	Microcontrolador (Arduino)	SBC (Raspberry Pi)	PLC Industrial / PC Industrial
Coste Aproximado	Bajo (<50€)	Medio (100€ - 300€)	Alto (>10.000€)

Tabla 1: Comparativa entre tecnologías de guiado: IR, Visión y AGV.

Tras el análisis de la Tabla 1 se puede concluir que la arquitectura de este proyecto se posiciona en la categoría central, buscando maximizar las capacidades de la *Visión Artificial* para emular comportamientos similar a un *AGV* (toma de decisiones, lectura de señales) utilizando hardware de consumo.

Capítulo 4

Análisis del problema

La navegación autónoma en entornos controlados ha evolucionado desde simples seguidores de línea basados en contraste hasta sistemas capaces de interpretar su entorno. Sin embargo, la implementación de vehículos que no solo sigan un camino, sino que entiendan el contexto del mismo (señales, cruces, zonas de velocidad), plantea desafíos significativos cuando se busca mantener un bajo coste y una alta accesibilidad educativa.

Este capítulo aborda la definición formal del problema, separando la necesidad operativa real de los desafíos técnicos que conlleva su implementación, así como los requisitos abstractos que cualquier solución propuesta debe satisfacer.

4.1. Definición del Problema

Para comprender el alcance del proyecto, es necesario distinguir entre el problema que se desea resolver en el mundo físico (Problema Real) y las dificultades de ingeniería que surgen al intentar resolverlo (Problema Técnico).

4.1.1. Problema Real

Como hemos visto anteriormente en este documento, el desarrollo de los sistemas empujados y el paradigma del Internet de las cosas (IoT) han revolucionado sectores como el de la industria o la robótica. Gracias al incremento de la capacidad de cómputo y el reducido consumo de muchos de los sistemas embebidos de hoy en día, se han abierto nuevas posibilidades para el desarrollo de proyectos de investigación.

El problema real consiste en la necesidad de dotar a un vehículo a escala de la capacidad de desplazarse de forma autónoma por un circuito preestablecido, respetando una serie de reglas de navegación visuales. El vehículo no solo debe mantenerse dentro de los límites del carril, sino que debe comportarse de manera inteligente ante estímulos visuales, simulando un entorno de tráfico simplificado.

El sistema debe resolver la incógnita de "¿Cómo navegar del punto A al punto B sin intervención humana, interpretando las señales visuales del suelo y reaccionando a ellas en tiempo real?". Esto implica que el vehículo debe dejar de ser un autómata reactivo para convertirse en un sistema con capacidad de decisión contextual.

4.1.2. Problema Técnico

La resolución del problema real descrito anteriormente se concreta en un desafío técnico principal: el diseño e implementación de un sistema embebido capaz de procesar información en tiempo real con recursos computacionales limitados. Abordar este desarrollo desde una perspectiva ingenieril requiere descomponer el proyecto en una serie de retos técnicos interdependientes que deben ser resueltos de manera secuencial e incremental.

Construir el prototipo funcional implica aplicar una metodología de desarrollo de sistemas empotrados que considere los siguientes apartados críticos:

- **Elección de componentes Hardware-Software:** El primer reto técnico consiste en la selección de los componentes adecuados para el proyecto. En esta elección entra en juego tanto componentes físicos como sensores, chasis, motores, etc... como software, lenguaje de programación, librerías, algoritmos, etc... Todo ello debe ser seleccionado tomando en cuenta factores como el coste, la capacidad de procesamiento, la facilidad de uso, el tiempo de respuesta, la compatibilidad con otros componentes... entre otros factores que influyan en el desempeño del sistema.
- **Integración y Abstracción:** El siguiente paso consiste en establecer una comunicación eficiente entre los componentes físicos (sensores infrarrojos, cámara, drivers de motores, etc...) y la lógica de control. Esto requiere el desarrollo de una capa de abstracción de hardware (HAL) o el uso de librerías que permitan desacoplar la lógica de navegación de los detalles de bajo nivel de los componentes electrónicos. El sistema debe ser capaz de leer múltiples sensores de distinta naturaleza de forma concurrente sin bloquear el ciclo principal de toma de decisiones.
- **Diseño de una arquitectura modular y escalable:** A diferencia de un robot sigue-líneas monolítico, el requisito de "navegación contextual" impone una arquitectura de software modular. El problema técnico reside en diseñar un sistema donde el módulo de "seguimiento de línea" pueda coexistir con módulos superiores de "interpretación de contexto" (visión por computador o detección de eventos). La arquitectura debe permitir que la complejidad del sistema crezca incrementalmente sin necesidad de reescribir el núcleo de control de movimiento.
- **Gestión de Recursos en Sistemas de Bajo Coste:** La capacidad de interpretación del entorno y la gestión de la toma de decisiones contextuales son tareas computacionalmente costosas. Dado que se trabaja con sistemas embebidos de bajo coste, existe una restricción técnica estricta en cuanto a capacidad de CPU, memoria y consumo energético. El reto técnico implica optimización de los algoritmos para que operen con una latencia mínima, garantizando que el vehículo pueda reaccionar a tiempo a los estímulos del entorno mientras mantiene una velocidad de avance adecuada.
- **Validación y Ajuste en Entorno Físico:** El problema técnico abarca la validación del sistema en un entorno real. A diferencia de una simulación puramente software, el despliegue físico introduce variables como el ruido en los sensores, la fricción del terreno o la respuesta no lineal de los motores. El sistema debe incluir mecanismos de calibración y ajuste (como la sintonización de controladores PID) que permitan absorber estas perturbaciones físicas para cumplir con los objetivos de navegación autónoma.

- **Experimentación:** Una vez validado el sistema, se debe proceder a la experimentación para evaluar su rendimiento en diferentes condiciones y escenarios. Aumentando la complejidad del circuito gradualmente para evaluar la robustez del sistema y determinar si es capaz de cumplir con los objetivos de navegación autónoma.
- **Documentación:** El último reto técnico consiste en documentar el proceso de desarrollo y los resultados obtenidos de manera clara y concisa. Para ello es importante ofrecer dos tipos de documentación. El primer tipo se trata de un manual de código donde queden reflejados la lógica y los procesos que hay detrás del funcionamiento del sistema. El segundo tipo de documentación se trata de un manual de usuario que permita a cualquier persona, independientemente de sus conocimientos previos, hacer uso del sistema sin mayor complicación.

4.2. Identificación de Requisitos

Para formalizar la solución al problema planteado, es necesario establecer un contrato técnico mediante la definición de requisitos. Estos requisitos actúan como puente entre la necesidad abstracta (navegación autónoma) y la implementación concreta.

Antes de enumerar los requisitos funcionales, es conveniente identificar los actores y el alcance de la interacción con el sistema:

- **El Vehículo (Sistema):** Actor principal que ejecuta las acciones de navegación y toma de decisiones.
- **El Entorno (Contexto):** Agente pasivo que provee la información (línea, señales, obstáculos) que el vehículo debe interpretar.
- **El Operador (Usuario):** Encargado de la puesta en marcha, configuración de modos y supervisión de seguridad.

El sistema se concibe como una “caja negra” que recibe estímulos visuales del entorno y responde sin intervención del operador, devolviendo acciones de movimiento físico. A continuación, se desglosan las funcionalidades específicas (RF) y las restricciones de calidad (RNF) que regirán el desarrollo.

4.2.1. Requisitos Funcionales (RF)

Los requisitos funcionales definen el comportamiento específico que el sistema debe exhibir en respuesta a entradas y condiciones determinadas. A continuación, se detallan las funciones críticas que el prototipo de navegación autónoma debe implementar para dar cumplimiento a los objetivos planteados.

Tabla 2: Requisitos funcionales (RF) del sistema.

Código	Descripción
RF01	El sistema debe ser capaz de controlar de forma independiente la velocidad y sentido de giro de los motores de tracción para permitir maniobras de avance, retroceso y giro sobre su propio eje.
RF02	El sistema debe implementar una capa de abstracción de hardware (HAL) que permita enviar comandos de movimiento (ej. "avanzar", "parar") desacoplados de la electrónica específica de los drivers de potencia.
RF03	El robot debe adquirir y procesar datos de los sensores pertinentes en tiempo real para determinar su posición relativa respecto a una línea guía marcada en la superficie.
RF04	El sistema debe ser capaz de detectar la presencia de marcas visuales o cambios cromáticos específicos en el entorno (contexto) con el uso de algún tipo de periférico o sensor.
RF05	El sistema debe ser capaz de detectar la presencia de señales o códigos durante su recorrido por el circuito en caso de que los haya.
RF06	El robot debe implementar un algoritmo de control en bucle cerrado (tipo PID o similar) que corrija la trayectoria del vehículo automáticamente para mantener el seguimiento de la línea.
RF07	El sistema debe disponer de un gestor de eventos capaz de interrumpir o modificar el comportamiento de seguimiento de línea (ej. detenerse, cambiar velocidad) al reconocer un estímulo contextual validado (RF04).
RF08	El vehículo debe comprender el significado de las señales o códigos detectados (RF05), priorizando esta acción sobre el seguimiento de línea.
RF09	El software debe permitir la activación o desactivación de módulos de comportamiento mediante configuración.
RF10	En caso de pérdida momentánea de la referencia visual, el sistema debe ejecutar protocolos de búsqueda o parada segura, evitando comportamientos erráticos.
RF11	El sistema debe calcular en tiempo real el error de posición (desviación lateral) respecto al centro de la línea en base a los datos obtenidos por los sensores (RF03) y corregir su dirección.
RF12	El sistema debe tener algún tipo de interfaz que permita al usuario interactuar con el sistema.

4.2.2. Requisitos No Funcionales (RNF)

Los requisitos no funcionales establecen las restricciones y condiciones de calidad bajo las cuales debe operar el sistema. Dado que este proyecto se enfoca en sistemas embebidos de bajo coste, las limitaciones de recursos hardware y energéticos son determinantes para el diseño.

Tabla 3: Requisitos no funcionales (RNF) del sistema.

Código	Descripción
RNF01	El sistema debe ser autónomo energéticamente, operando mediante baterías recargables y optimizando el consumo para permitir ciclos de prueba funcionales sin necesidad de alimentación externa constante.
RNF02	La arquitectura software debe estar orientada en la reusabilidad y mantenibilidad del código, permitiendo la adición de nuevos sensores o lógica de control sin requerir la refactorización completa del código base.
RNF03	El sistema debe implementarse sobre hardware comercial de propósito general y fácil adquisición (tipo Arduino, ESP32 o Raspberry Pi), evitando soluciones propietarias cerradas o discontinuadas.
RNF04	Todo el código fuente y la documentación de diseño generada deben ser publicados en un repositorio público bajo una licencia de software libre (Open Source) para fomentar la colaboración.
RNF05	El sistema de procesamiento debe operar con una latencia lo suficientemente baja para garantizar una respuesta en tiempo real ante el trazado del recorrido a la velocidad establecida.
RNF06	El diseño físico y las conexiones electrónicas deben estar documentados de tal forma que el prototipo pueda ser reproducido por terceros con conocimientos básicos de electrónica.
RNF07	La instalación del sistema operativo y las librerías necesarias debe ser documentada de tal forma que el usuario pueda replicar el entorno de desarrollo.
RNF08	Los algoritmos de percepción deben ser tolerantes a variaciones moderadas de iluminación y no depender de condiciones de laboratorio perfectas.
RNF09	Los componentes hardware del sistema deben de ser escogidos teniendo en cuenta el bajo coste de estos y su facilidad de obtención.

4.3. Restricciones

El desarrollo de cualquier sistema de ingeniería es un proceso de compromiso entre lo ideal y lo posible. Para este proyecto, la solución final no es arbitraria, sino el resultado directo de aplicar una serie de restricciones estrictas sobre el espacio de diseño.

A continuación, se detallan los factores estratégicos, de entorno y de datos que han forzado la elección de las tecnologías específicas de hardware y software empleadas.

4.3.1. Factores Estratégicos

Estos factores derivan de la naturaleza académica del proyecto y definen el marco de trabajo.

- **Temporalidad (Time-to-Market):** La naturaleza de un Trabajo de Fin de Grado impone un plazo de entrega inamovible. Esto prioriza drásticamente la velocidad de

desarrollo sobre la optimización extrema del código. Se requieren lenguajes de alto nivel y librerías preexistentes para evitar reinventar la rueda en tareas de bajo nivel (drivers, protocolos).

- **Presupuesto (Coste):** El requisito de "Bajo Coste" limita el hardware a componentes comerciales genéricos. Se descartan sensores industriales (LiDAR, cámaras estereoscópicas) en favor de sensores estándar y módulos de electrónica de consumo.
- **Accesibilidad y Replicabilidad:** La solución debe poder ser reproducida en un entorno educativo, lo que obliga a utilizar tecnologías de Código Abierto (Open Source) y hardware de fácil adquisición.

4.3.2. Factores de Datos

Las condiciones físicas y lógicas donde operará el robot imponen las siguientes limitaciones:

- **Entorno Físico Controlado:** Se asume una superficie plana y condiciones de iluminación artificiales (interiores). Esto elimina la necesidad de sistemas de suspensión mecánica complejos o cámaras con rango dinámico extremo (HDR).
- **Conectividad y Volumen de Datos:** Aunque se dispone de conexión Wi-Fi, el ancho de banda es un recurso compartido y limitado. No es viable transmitir vídeo en alta resolución a un servidor externo para su procesamiento (Cloud Computing).
- **Modelo de Computación:** Como consecuencia del punto anterior, se define una restricción de arquitectura: **todo el procesamiento crítico (visión y control) debe ser local**. El robot debe ser autónomo incluso si la red falla, utilizando la Wi-Fi únicamente para telemetría ligera y control de inicio/parada.

Capítulo 5

Diseño

En esta parte del documento se describe el diseño del sistema propuesto. Tras abordar las restricciones de diseño que debe cumplir el sistema propuesto, se justificarán las decisiones tomadas en base a dichas restricciones y a los requisitos establecidos. Posteriormente, se procederá a describir el diseño de la parte estática del sistema, el de la parte dinámica y la estructura del software.

5.1. Diseño Hardware

5.1.1. Justificación de las Decisiones de Diseño Hardware

Una vez establecido todo aquello que debe y no debe hacer el sistema, además de las condiciones que debe cumplir en su desarrollo, se procede a justificar las decisiones tomadas en el transcurso del desarrollo del sistema.

Selección del Sensor

Al comienzo del desarrollo de este prototipo, se optó por un sistema que estuviera conformado por un conjunto de componentes similares a los sigue-líneas tradicionales, es por ello que se usó en un inicio un sensor infrarrojo (IR) para la detección de la línea de seguimiento.

En un inicio la idea era usar el sensor infrarrojo para la detección de la línea de seguimiento y una cámara para la detección de señales y/o códigos. Pero al no cumplir con el requisito **RF-04** (detección marcas visuales o cambios cromáticos), se decidió delegar la detección de la línea de seguimiento a la cámara. Esta decisión permitió un gran abanico de posibilidades en el desarrollo del sistema, ya que se podía detectar la línea y el color de esta, haciendo que la adaptabilidad del sistema fuera mayor, cumpliendo de esta forma con los requisitos **RF-04** y **RF-05**.

Selección de la Unidad de Procesamiento:

Para este proyecto se requiere de una unidad de procesamiento que fuera capaz de gestionar el cálculo del error en la detección de la línea en tiempo real, además del procesamiento de imágenes para la detección y lectura de señales o códigos como se establece en el re-

quisito **RF-08**. Todo este trabajo sumado al manejo de los actuadores (servos) requieren de un procesador con una gran cantidad de potencia.

Con el fin de seleccionar la unidad de procesamiento adecuada, se evaluaron tres alternativas basándose en los requisitos de potencia de cálculo, capacidad de gestión de vídeo y familiaridad con su uso: sistemas con cámara integrada (ESP32-CAM), microcontroladores estándar (Arduino) y ordenadores de placa reducida (Raspberry Pi).

Opción A: Microcontroladores (ESP32 / ESP32-CAM)

Se analizó la viabilidad de utilizar un SoC ESP32-CAM debido a su bajo coste y consumo energético. Aunque esta plataforma permite realizar streaming de vídeo y tareas básicas de control, presenta limitaciones severas para la navegación autónoma contextual:

- **Limitación de Memoria:** Con apenas 520 KB de SRAM interna, el procesamiento de imágenes mediante matrices (como hace OpenCV) se vuelve inviable a resoluciones superiores a QQVGA sin usar memoria PSRAM externa, lo que reduce drásticamente los FPS.
- **Ausencia de Sistema Operativo Completo:** Al carecer de un núcleo Linux, la gestión de múltiples hilos (Websockets + Visión + Control) depende de FreeRTOS, aumentando la complejidad del desarrollo y limitando el uso de librerías de alto nivel como NumPy o Flask.

Opción B: Microcontroladores Estándar (Arduino Uno / Mega)

Es la plataforma más común en robótica educativa debido a su robustez y simplicidad para el control de motores (PWM) y lectura de sensores analógicos. Sin embargo, fue descartada como controlador principal por una limitación crítica:

- **Incapacidad de Procesamiento de Imagen:** Con una velocidad de reloj de 16 MHz y apenas 2 KB de SRAM (en el modelo Uno), un Arduino no tiene la potencia necesaria para el procesamiento de imágenes, haciendo imposible el uso de OpenCV.
- **Dependencia Externa:** Para usar visión con Arduino, sería necesario transmitir el vídeo a un ordenador externo para su procesamiento, introduciendo latencia y dependencia de una red Wi-Fi estable, rompiendo el principio de autonomía embarcada en el requisito funcional **RF-03**.

Opción C: Raspberry Pi 3 Model B+ (Seleccionada)

Representa el equilibrio óptimo entre potencia, coste y facilidad de desarrollo. Su arquitectura ARM Cortex-A53 de cuatro núcleos permite ejecutar un sistema operativo completo (Raspbian/Linux), lo cual es indispensable para la arquitectura multiproceso propuesta en este TFG.

Si bien es la elección de mayor coste, ofrece la facilidad de trabajar con una unidad potente y compacta con acceso a gran cantidad de librerías y herramientas útiles para el desarrollo de software.

Esta elección permite:

- Ejecutar un Sistema Operativo completo (Linux) para gestionar la pila de red y periféricos.
- Disponer de potencia suficiente (CPU 1.4GHz, 1GB RAM) para algoritmos de visión.
- Mantener el coste dentro del rango aceptable.

Con la elección de una Raspberry Pi se cumplen los requisitos no funcionales **RNF-03**, **RNF-05** y **RNF-09**. Además, gracias a esta elección se puede ejecutar algoritmos capaces de cumplir con muchos de los requisitos restantes del proyecto, que de elegir un microcontrolador no sería posible o se hubiera visto muy limitado.

Tabla Comparativa

La Tabla 4 resume las métricas técnicas que justifican la decisión.

Característica	Arduino Uno	ESP32-CAM	Raspberry Pi 3B+
Tipo	Microcontrolador	SoC (Micro)	SBC (Ordenador)
Procesador	16 MHz (8-bit)	240 MHz (32-bit)	1.4 GHz (64-bit Quad)
Memoria RAM	2 KB	520 KB + PSRAM	1 GB (1.000.000 KB)
Visión Artificial	Inviabile	Limitada	Nativa (OpenCV)
Lenguaje	C++ (Wiring)	C++ / MicroPython	Python 3 / C++
Sistema Operativo	Ninguno	FreeRTOS	Linux (Raspbian)

Tabla 4: Comparativa técnica de plataformas de control.

Selección del Lenguaje: Python vs. C++

A la hora de elegir el lenguaje para la unidad de procesamiento se tuvo en consideración varios lenguajes como C o C++, pero al final se optó por Python. Debido a que, aunque C++ ofrece un rendimiento superior y una gestión de memoria manual ideal para sistemas embebidos, se selecciona **Python 3**, porque permite un prototipado mucho más rápido y una integración trivial con la librería de visión OpenCV. Dado que la velocidad del vehículo físico está limitada por los motores y la seguridad, la latencia introducida por el intérprete de Python es un compromiso aceptable a cambio de la mantenibilidad y legibilidad del código, cumpliendo de esta forma con los requisitos **RNF-02** y **RNF-05**.

5.1.2. Especificaciones Hardware

Tras justificar la toma de decisiones de los principales componentes del sistema, se procede a enumerar y detallar todos los componentes electrónicos que conforman el prototipo con el fin de tener una visión general del hardware usado.

Plataforma de Procesamiento: Raspberry Pi

Como se puede observar en el cuadro 5 la unidad de procesamiento escogida ha sido una **Raspberry Pi 3 Model B+** cuenta con un procesador ARM Cortex-A53 de cuatro núcleos a 1.4GHz [26]. Esta arquitectura es requisito indispensable para ejecutar el sistema

operativo Linux y las librerías de visión artificial (OpenCV) necesarias para la navegación contextual.

Una de las ventajas de esta placa es que utiliza el puerto dedicado CSI (*Camera Serial Interface*), que ofrece un ancho de banda muy superior al USB, reduciendo la latencia en la captura de fotogramas, un factor crítico para la conducción autónoma a alta velocidad.

5.1.3. Sistema de Actuadores y Sensores

El vehículo, basado en el kit *Adept PiCar-B*, integra los siguientes subsistemas controlados mediante la interfaz GPIO y el bus I2C:

1. **Tracción (Motor DC):** Un motor de corriente continua situado en el eje trasero proporciona la propulsión. Su velocidad se regula mediante señales PWM (*Pulse Width Modulation*) gestionadas por el controlador de motores (driver TB6612 o L298N integrado en el HAT), lo que permite un ajuste fino de la aceleración y frenado.
2. **Dirección (Servomotor):** Un servomotor controla el ángulo de las ruedas delanteras. A diferencia de los sistemas de dirección diferencial (tipo tanque), este diseño imita la geometría Ackerman [27] de un automóvil real, lo que requiere un cálculo preciso del ángulo de giro en función de la curvatura de la línea.
3. **Cabezal de Cámara (Pan/Tilt):** Dos servomotores adicionales permiten mover la cámara en los ejes horizontal (*pan*) y vertical (*tilt*). En el diseño de software, estos servos se mantienen fijos o se estabilizan para garantizar que la “Región de Interés” (ROI) de la visión se mantenga constante sobre la carretera. Estos servos también son esenciales en la detección de las señales (**RNF-05**) al permitir el movimiento de la cámara en el eje horizontal y vertical.
4. **Sensor de Visión:** Se emplea el módulo de cámara oficial de Raspberry Pi (v2 o compatible) [28], configurado para capturar vídeo a una resolución de 640x480 píxeles a 60 FPS, maximizando la fluidez temporal sobre la resolución espacial y aprovechando el puerto CSI de la Raspberry Pi.
5. **Placa de Expansión:** Adept Motor HAT V2.0, conectada a la Raspberry Pi a través de los pines GPIO. La función principal de esta placa es actuar como *punto de hardware* permitiendo que el microcontrolador pueda controlar dispositivos de alto consumo de corriente, como los motores, utilizando únicamente señales digitales de control. En el cuadro 5 se pueden observar algunas de sus especificaciones técnicas.
6. **Batería:** Para la alimentación del robot se usan dos baterías de iones de litio (Li-ion) de 3.7V y 2000mAh cada una, conectadas en serie para proporcionar una tensión de salida de 7.4V. Estas baterías se conectan a través del portabaterías, que va conectado directamente a la placa de expansión **Adept Motor HAT V2.0**.
7. **Leds frontales:** El kit utilizado incorpora dos LEDs frontales que se encienden cuando el vehículo está en movimiento. Aunque no son esenciales para el funcionamiento del robot, su presencia puede ser útil para la depuración y el monitoreo del estado del vehículo. Debido a que se le ha atribuido un color a cada acción del vehículo, se puede identificar fácilmente el estado del vehículo, por ejemplo: se le ha atribuido el color amarillo a los leds cuando el vehículo está en el modo de se-

guimiento de línea. Estos leds se encuentran conectados al *Adept Motor HAT V2.0* como se observa en la figura 1.

En la figura 1 se puede ver como se conectan todos los componentes mencionados en el vehículo a la Raspberry Pi gracias a la placa de expansión Adept Motor HAT V2.0.

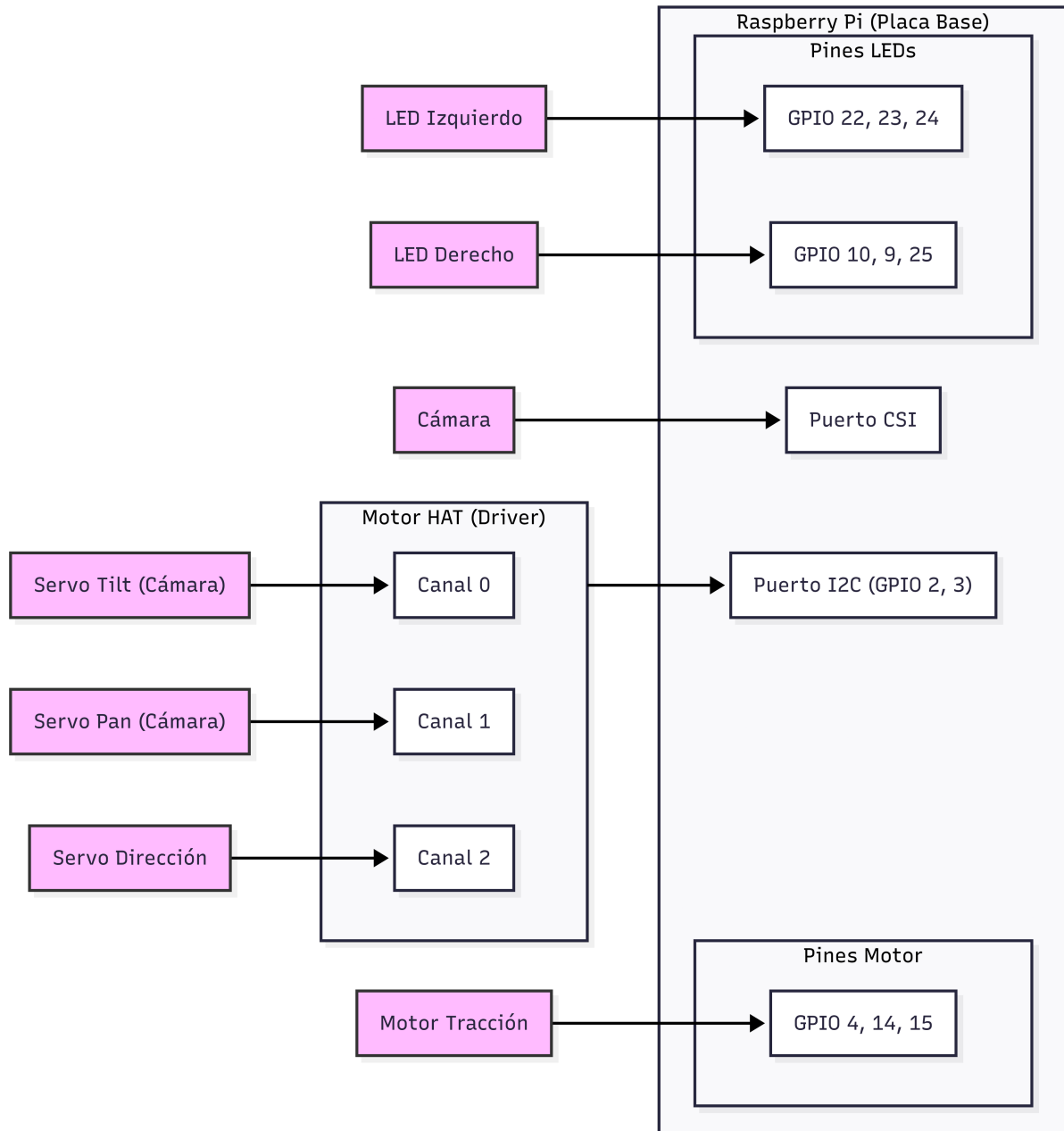


Figura 1: Esquema de conexiones.

5.1.4. Componentes del Hardware

A continuación se detallan las especificaciones técnicas de los principales componentes electrónicos antes mencionados.

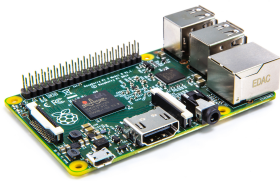
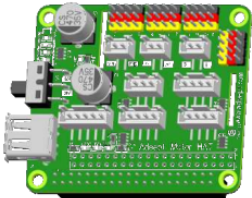
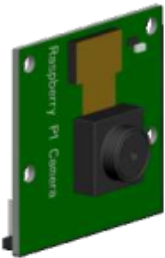
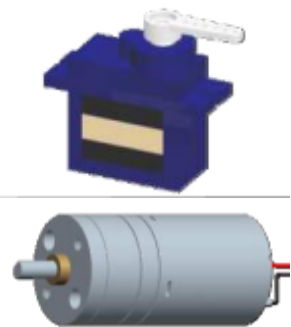
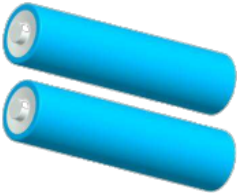
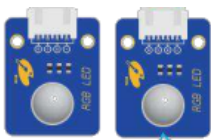
Componente	Descripción Técnica y Función
	Raspberry Pi 3 Model B+ Es el núcleo de procesamiento del sistema. <i>Specs:</i> CPU Quad Core 1.4GHz, 1GB RAM. <i>Función:</i> Ejecuta el sistema operativo Linux, el servidor Web y los algoritmos de visión artificial con OpenCV.
	Adept Motor HAT V2.0 Placa de expansión conectada vía GPIO. <i>Chipset:</i> Integra el controlador PWM PCA9685 (I2C) para los servos y el Puente H TB6612 para el motor DC. <i>Función:</i> Distribución de energía y control de actuadores.
	Raspberry Pi Camera Module v2 Sensor de imagen Sony IMX219 de 8 Megapíxeles. <i>Interfaz:</i> Puerto CSI (Camera Serial Interface). <i>Función:</i> Captura de vídeo de baja latencia para el streaming y procesamiento de visión.
	Actuadores <i>Dirección:</i> Servo MG90S de alto torque. <i>Tracción:</i> Motor DC con reductora (Gear Motor). <i>Función:</i> Ejecución física de las órdenes de movimiento (giro y avance) enviadas por el software.
	Baterías 18650 (x2) Fuente de alimentación de alta capacidad (3.7V x 2). <i>Función:</i> Proporcionan la corriente de alta descarga necesaria para mantener estables los servos y la Raspberry Pi simultáneamente.
	Módulos LED RGB (x2) Diodos emisores de luz tricolor conectados a los pines GPIO a través de la placa de expansión. <i>Control:</i> Librería gpiozero (PWM software). <i>Función:</i> Interfaz visual hombre-máquina (HMI). Indican estados (inicio, espera) y modos de operación.

Tabla 5: Componentes hardware del robot.

5.1.5. Diagrama de conexiones del robot

En la figura 2 se puede ver el diagrama de conexiones del robot. Donde se puede apreciar las conexiones de todos los componentes con la placa de expansión Adept Motor HAT V2. Por otra parte, la cámara se conecta a la Raspberry Pi a través del puerto CSI, dicha conexión se puede ver en la figura 3.

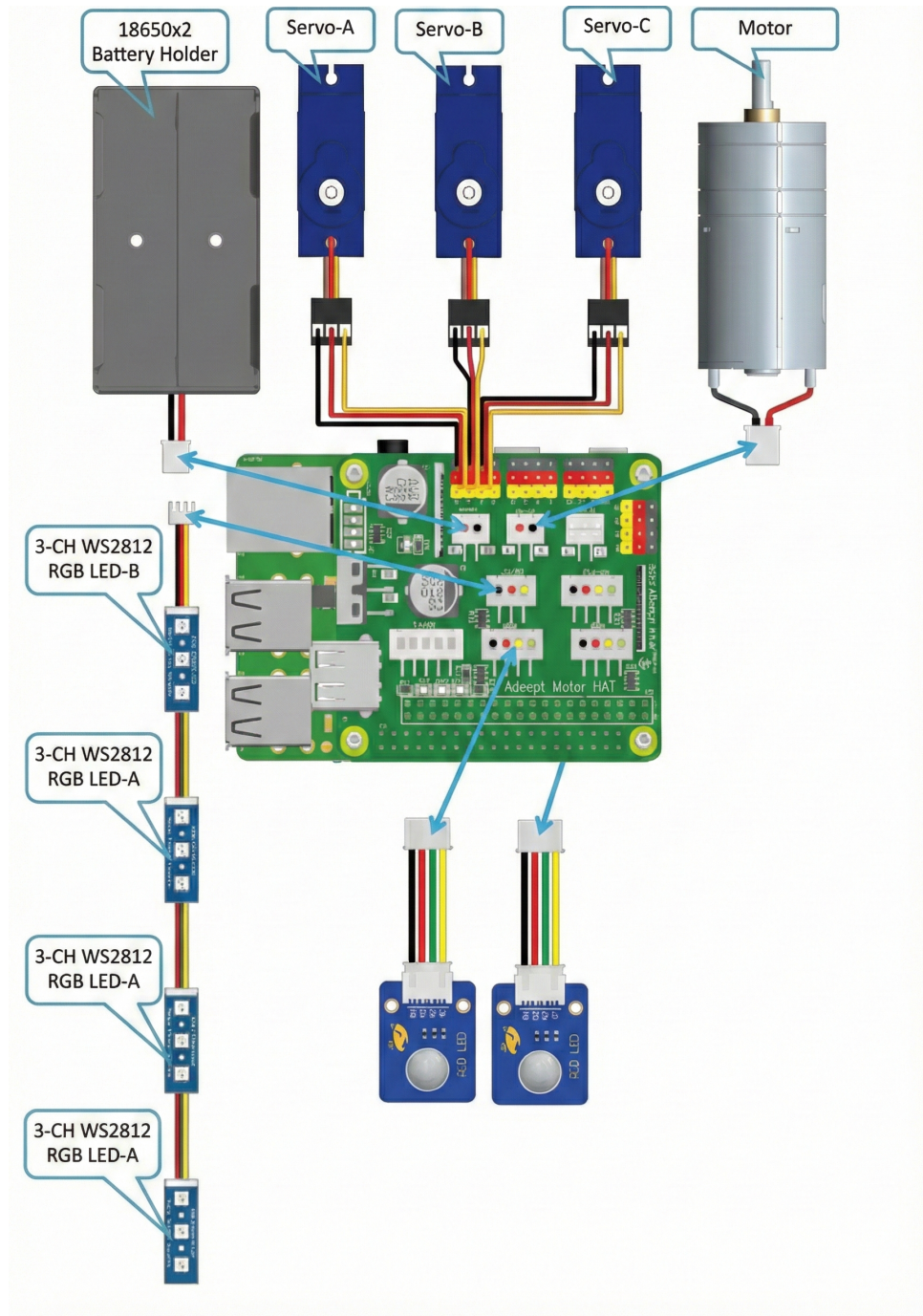


Figura 2: Diagrama de conexiones del robot.

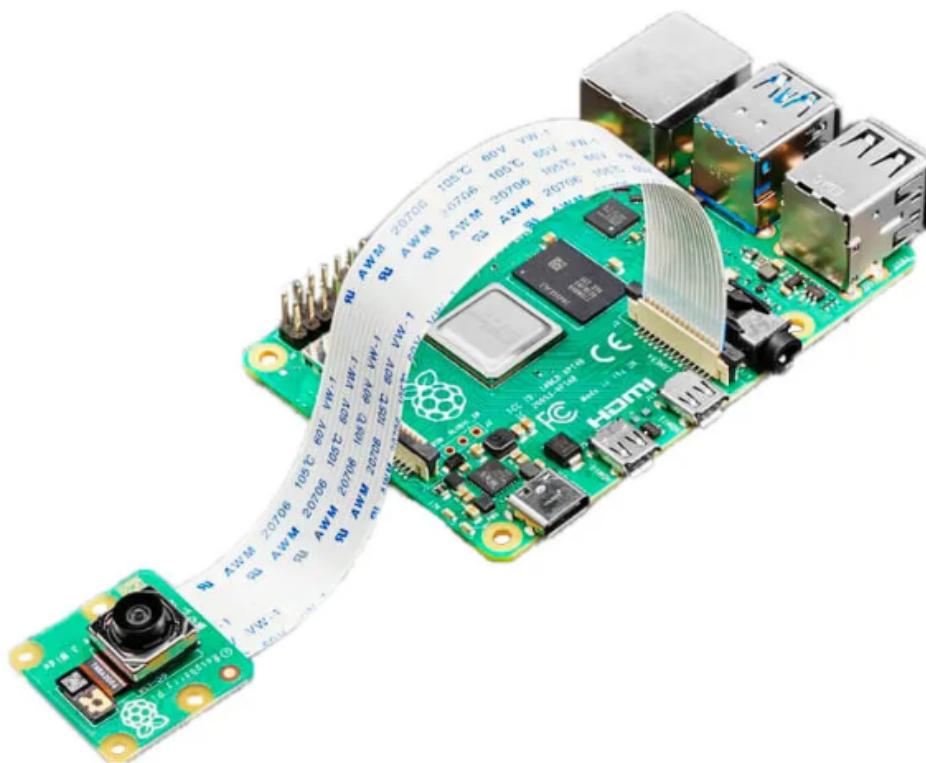


Figura 3: Diagrama de conexión de la cámara.

5.2. Diseño Software

Tras haber tratado los componentes hardware utilizados en el desarrollo del prototipo, a continuación se detallarán los aspectos del software utilizados en el mismo.

Durante la elaboración del software del sistema se ha tenido en tanto los componentes físicos del vehículo, como los requisitos funcionales que el sistema debe cumplir.

Antes de empezar a hablar de los diferentes archivos que componen el software del sistema, es importante mencionar que todo el código fuente del sistema ha sido recogido en el repositorio de GitHub [29], con el objetivo de cumplir con el requisito no funcional **RNF-04**.

5.2.1. Modelo de Datos

El diseño estático define la estructura modular del software, identificando los componentes principales, sus responsabilidades y las relaciones de dependencia entre ellos. Para este proyecto, se ha optado por una Arquitectura por Capas (Layered Architecture), lo que permite desacoplar la lógica de navegación de los detalles físicos del hardware, cumpliendo con el requisito de mantenibilidad (RNF-02). El código, desarrollado en Python 3, se organiza en cuatro niveles de abstracción jerárquica, donde cada capa solo interactúa con sus adyacentes y entre los elementos de una misma capa.

Arquitectura por capas

Como se puede ver en la figura 4, el proyecto está organizado en cuatro capas, cada una con sus propios componentes. A continuación se detallará cada una de las capas:

- **Capa 1: Hardware Físico:** La capa física está compuesta por los componentes físicos del robot, como la cámara CSI, los LEDs GPIO, el PCA9685 y el TB6612 que hemos visto anteriormente en el cuadro 5.
- **Capa 2: Abstracción de Hardware (HAL):** Esta capa es la más baja dentro del software y se encarga de abstraer el hardware físico del robot, cumpliendo con el requisito **RF-01**.
- **Capa 3: Visión y Percepción:** En esta capa se produce todo el ciclo de la imagen, desde su captura hasta su procesamiento y puesta a punto para el uso de los componentes de las capas superiores.
- **Capa 4: Lógica de Control:** Esta capa alberga toda la lógica del funcionamiento del coche. Gestionando el control de este.
- **Capa 5: Aplicación (Servidor y cliente):** Esta capa es la más alta dentro del software y se encarga de la conexión con el usuario y proporcionar una interfaz para controlar el robot como dicta el requisito **RF-12**.

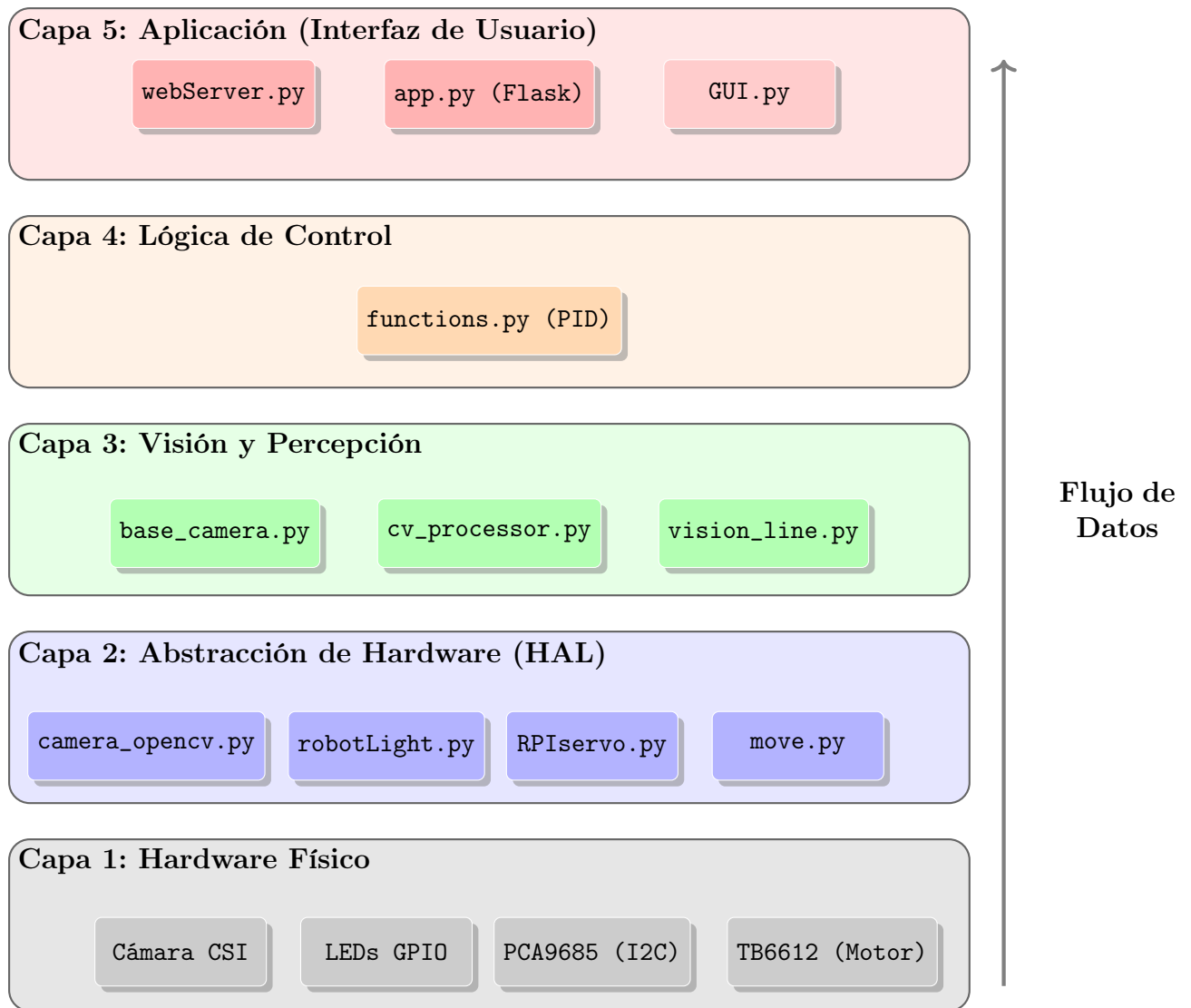


Figura 4: Diagrama de la Estructura por Capas del Sistema.

Diagrama de Clases

Una vez conocida la jerarquía de los diferentes archivos, se procede a explicar el funcionamiento general del sistema a través de las clases que lo conforman.

La figura 7 muestra una perspectiva general de las clases que componen el sistema, donde se puede ver la jerarquía de los diferentes archivos y las relaciones entre ellos. Con el fin de facilitar una comprensión sobre el diseño de cada clase, se aborda una descripción de cada una de ellas en la tabla 7.

A continuación se va a definir el funcionamiento de la lógica del sistema, para ello se seguirá el orden de las capas de forma ascendente, empezando por la capa 2 (más cercana al hardware) y terminando por la capa 5.

Capa 2: Abstracción de Hardware (HAL)

Esta capa aísla la complejidad electrónica del resto del sistema. Aquí se definen las primitivas de movimiento y actuación, interactuando directamente con los puertos GPIO e

I2C de la Raspberry Pi.

Anteriormente en el cuadro 5 hemos visto los distintos componentes electrónicos que son parte del robot, para el funcionamiento de estos componentes ha sido necesaria su implementación a través del código, con el objetivo de poder controlar su funcionamiento de forma sencilla por el resto de capas. A continuación se va a describir como ha sido dicha implementación:

- **RPIservo.py** : Este archivo es el encargado de gestionar los servomotores utilizando para ello el chip PCA9685. a través del protocolo I2C [30]. Este componente se encarga de inicializar y configurar el controlador PWM PCA9685 a una frecuencia operativa de 50 Hz, utilizando las librerías de `adafruit_circuitpython` para establecer la comunicación de bajo nivel. Su función principal es exponer métodos de alto nivel que permiten posicionar cualquiera de los 16 canales de servo disponibles en un ángulo específico entre 0 y 180 grados, implementando internamente mecanismos de seguridad ("clamping") para validar los rangos de entrada y manejo de excepciones para garantizar la estabilidad del sistema frente a errores de comunicación. Además, el módulo incluye rutinas de limpieza y liberación de recursos, asegurando que la gestión de los actuadores sea transparente y desacoplada de la lógica de control principal del robot.

En la figura 5 se puede ver el proceso de inicialización de los servomotores.

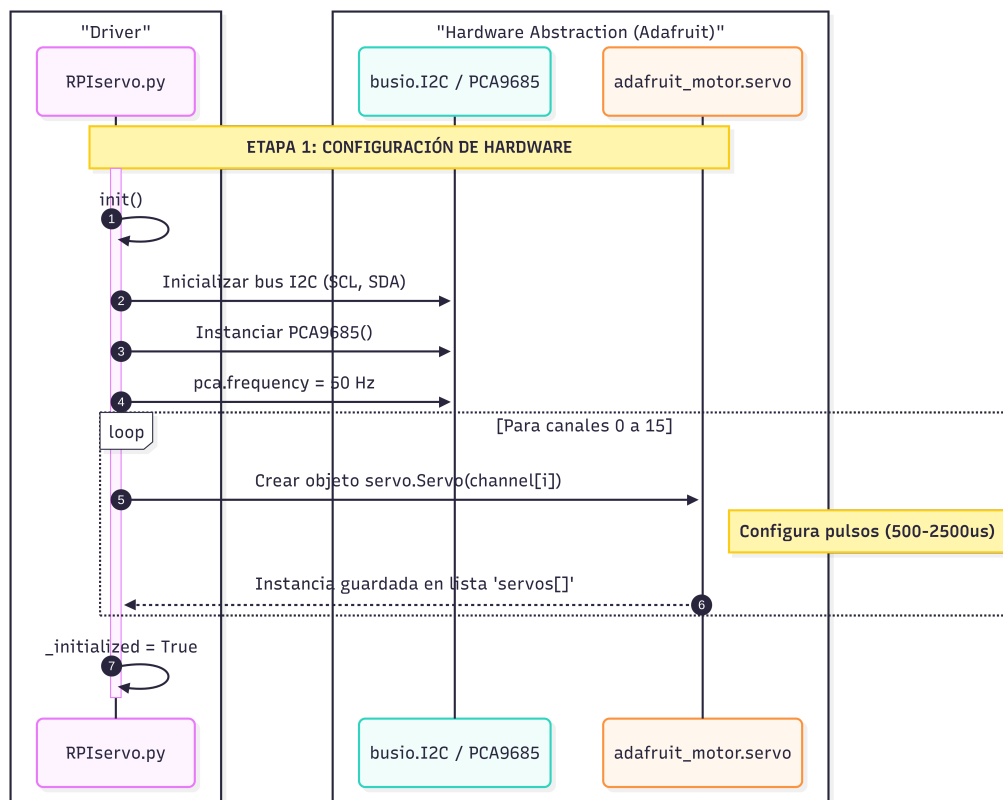


Figura 5: Secuencia de inicialización del bus I2C y servos.

- **move.py** : Es el encargado de gestionar el sistema de tracción trasera del robot

mediante el control directo del motor de corriente continua, actuando como interfaz lógica para el puente H a través de la librería **GPIO** de la Raspberry Pi. Este módulo se encarga de configurar los pines del procesador necesarios para la dirección (IN1, IN2) y la velocidad (Enable), implementando una señal de modulación por ancho de pulsos (PWM) a 1000 Hz que permite ajustar la potencia de tracción de forma granular entre 0 y 100. Proporciona funciones de alto nivel (**forward**, **backward**, **stop**) que abstraen la complejidad eléctrica de la inversión de polaridad y el ciclo de trabajo, garantizando un movimiento fluido del vehículo y asegurando la integridad del hardware mediante rutinas de limpieza que liberan los recursos GPIO al finalizar la ejecución del programa.

- **robotLight.py** : Este módulo proporciona la interfaz de control para el sistema de iluminación del robot, gestionando específicamente los módulos LED RGB frontales mediante la librería **gpiozero**. Esta clase encapsula la configuración de los pines GPIO asociados a los canales rojo, verde y azul de cada foco (izquierdo y derecho), permitiendo la manipulación unificada de su estado lumínico. A través de métodos como **front_color**, el módulo abstrae la mezcla de colores mediante un mapeo predefinido de valores binarios, facilitando la selección de estados visuales (como rojo, azul o blanco) sin necesidad de gestionar directamente las señales eléctricas. Además, implementa mecanismos de seguridad y limpieza de recursos (**cleanup**) para apagar los componentes al finalizar la ejecución, y mantiene compatibilidad con versiones anteriores del software mediante métodos vacíos que evitan errores en llamadas a funciones obsoletas.

En la Tabla 6 se pueden ver los pines de cada led y el color al que están asociados.

LED	Rojo (R)	Verde (G)	Azul (B)
Izquierdo	GPIO 22	GPIO 23	GPIO 24
Derecho	GPIO 10	GPIO 9	GPIO 25

Tabla 6: Asignación de pines GPIO para los LEDs frontales.

- **camera_opencv.py** : Este módulo implementa el sistema de visión por computador del robot actuando como puente entre el hardware de captura y la lógica de procesamiento. Utilizando un patrón Singleton, gestiona el acceso exclusivo a la cámara mediante la librería **picamera2** (configurada a 640x480 y 60 FPS) para evitar conflictos de recursos, y se encarga del bucle principal de adquisición de imágenes. Se especificará más sobre su funcionamiento más adelante.

Capa 3: Visión y Percepción

Sobre la capa de hardware se construye la capacidad sensorial del robot. La Figura 6 ilustra la relación entre las clases **Camera** (Tabla 15 y 16), **BaseCamera** (Tabla 14), **CameraEvent** (Tabla 13 y **CVProcessor** (Tabla 17 y 18).

Esta capa abarca desde la obtención de imágenes desde la cámara, hasta el procesamiento de visión artificial y la visualización de resultados.

- **Obtención de la imagen (clase `Camera`):** El proceso de obtención de la imagen comienza en el módulo `camera_opencv.py` el cual, como hemos mencionado antes, es el encargado de la adquisición de imágenes desde la cámara mediante la librería `picamera2`. El patrón *Singleton* implementado en la clase **`Camera`** garantiza que solo haya una instancia de la cámara en todo el sistema, lo que evita conflictos de recursos y permite un acceso exclusivo a la cámara.

El ciclo de captura funciona mediante un generador continuo de fotogramas (**`frames()`** tabla 16). En cada iteración, el sistema captura un array de imagen cruda en formato RGB directamente desde el hardware y lo convierte a formato BGR, que es el estándar utilizado por OpenCV. Inmediatamente después, este frame se pasa a la clase **`CVProcessor`** para su procesamiento a través de la función `draw_elements_on_frame()` (tabla 18).

- **Procesamiento de la imagen (clase `CVProcessor`):** Una vez obtenida la imagen y estando en el modo de operación **`TrackLine`**, el sistema pasa a la clase **`CVProcessor`**. En lugar de procesarla directamente, se crea una copia de la imagen y la gestión de la imagen se traslada a un proceso hijo (`_vision_worker_main`), este proceso es el encargado de enviar a las imágenes a través de una cola (*Queue*) al módulo **`vision_line.py`** para su posterior procesamiento.

Mientras la imagen es procesada, el proceso hijo espera en la cola de salida (`_qout`) el retorno de los datos de una imagen enviada anteriormente, que ya haya sido procesada por el módulo **`vision_line.py`**.

Cuando el proceso hijo recibe los datos de una imagen procesada, los envía al proceso padre, que a su vez lo envía a la clase **`Camera`**, parte de esos datos son las modificaciones (overlays) generados por el módulo **`vision_line.py`**. Por otro lado, el proceso padre (**`CVProcessor`**) envía un diccionario (*state*) con los valores de los parámetros medidos por el módulo **`vision_line.py`** a la clase **`Functions`**.

- **Procesamiento de la imagen (módulo `vision_line.py`):** Este proceso ocurre cuando el módulo **`vision_line`** recibe una imagen de la clase **`CVProcessor`**. El procesamiento de la imagen se basa en dividir la imagen capturada en varias franjas horizontales para analizar la carretera por secciones, desde lo que está inmediatamente frente al robot hasta lo que se aproxima en la distancia. Para cada una de estas franjas, el sistema genera simultáneamente varias máscaras binarias: utiliza rangos de color HSV para detectar los distintos colores preestablecidos que puede tener la línea (completando el requisito funcional **RF-04**), y algoritmos de contraste adaptativo (como Otsu) para detectar líneas negras sobre cualquier superficie clara. Estas máscaras se limpian de ruido mediante operaciones morfológicas y se analizan para encontrar contornos geométricos que parezcan líneas válidas, descartando formas irregulares o demasiado pequeñas. Una vez identificados los candidatos en cada banda, el algoritmo selecciona el “ganador” basándose en el tamaño y la consistencia con las detecciones anteriores, aplicando además un suavizado temporal (Media Móvil Exponencial) para evitar cambios bruscos por vibraciones. Finalmente, calcula la posición horizontal (cx) de la línea detectada en cada altura y determina el error respecto al centro de la imagen; este conjunto de errores y datos de color se empaqueta en un diccionario de estado que es lo único que se envía a la lógica de control, permitiendo al robot ajustar su dirección de forma suave y anticipada, y cumpliendo de esta forma el requisito **RF-03**.

- **Envío de imagen:** El envío de la imagen es gestionado por la clase **BaseCamera**, la cual actúa como un gestor inteligente de tráfico para el vídeo, diseñado para resolver el problema de conectar múltiples usuarios a una única cámara física. En lugar de que cada cliente web intente acceder al hardware directamente (lo cual daría error), esta clase ejecuta un único hilo de fondo que captura las imágenes constantemente y las deposita en una variable compartida. Utiliza la clase (**CameraEvent**) como sistema de eventos de sincronización para avisar simultáneamente a todos los clientes conectados cada vez que hay una nueva foto lista, permitiendo así que muchos usuarios vean el mismo flujo de vídeo fluido sin duplicar el trabajo de procesamiento ni bloquear el dispositivo.

En la figura 6 se representan las clases que conforman la capa 3 y sus relaciones entre sí:

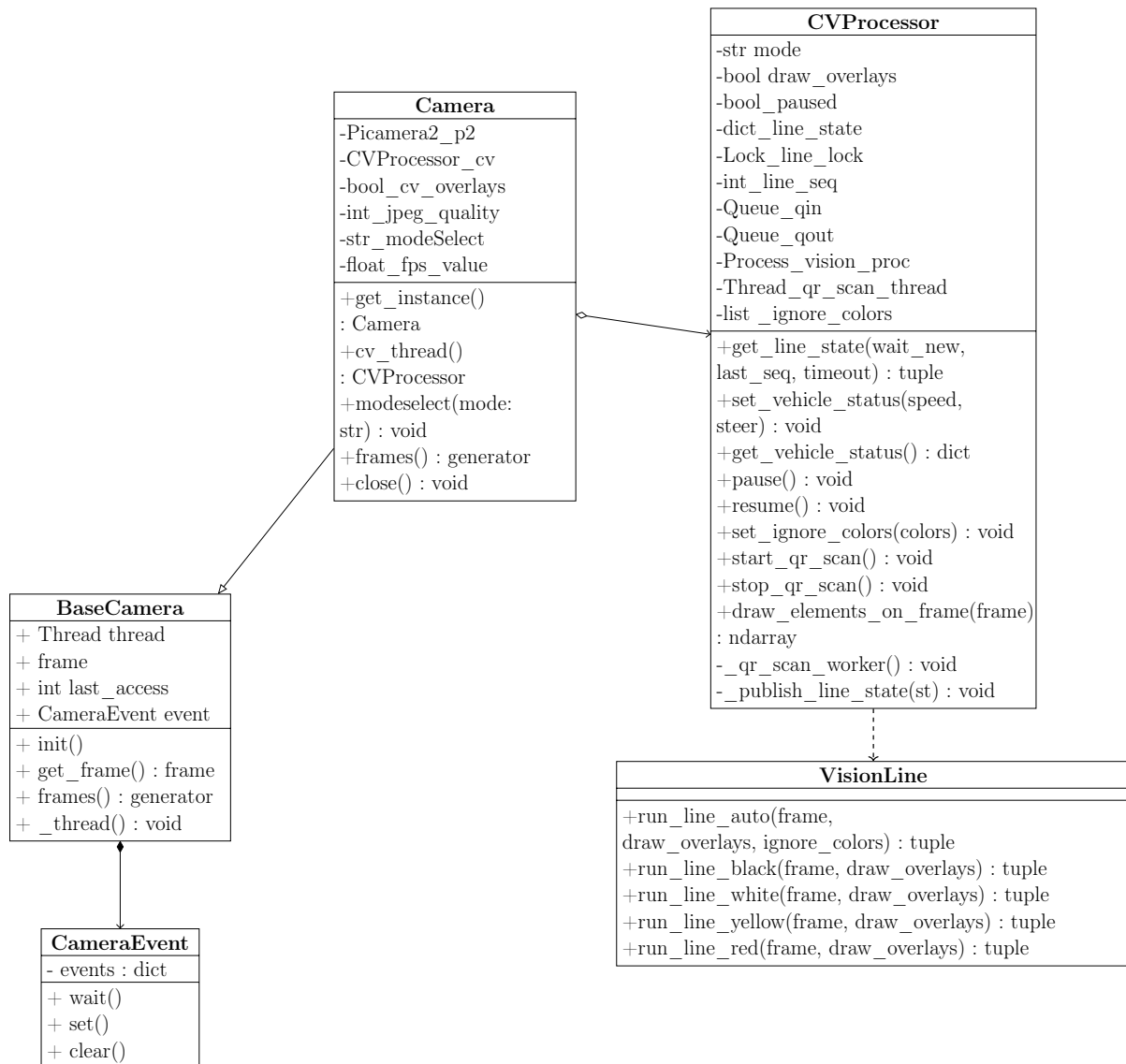


Figura 6: Diagrama de clases de la capa de Visión y Percepción.

Capa 4: Lógica de control

Esta capa está compuesta por un único módulo **Functions.py**. En este módulo reside toda la lógica de control del movimiento del robot, actuando como el piloto automático

del robot, coordinando la inteligencia de visión con los sistemas motrices. Su función central es traducir los datos abstractos recibidos del procesador de visión (CVProcessor) en comandos físicos para los servos y motores. Dentro de su bucle principal, gestiona una máquina de estados compleja que no solo reacciona a la posición de la línea, sino que también interpreta señales del entorno como códigos QR para tomar decisiones de alto nivel (cambiar de buscar líneas blancas a amarillas, detenerse o girar en una intersección).

El corazón de este sistema es el método `trackLineProcessing`. En cada ciclo, este método descarga el último reporte de visión, que contiene la posición de la línea en múltiples distancias (cerca, media, lejos). Con estos datos, ejecuta un algoritmo de control robusto: primero aplica lógica de recuperación si ha perdido la línea (iniciando maniobras de búsqueda o reversa como establece el requisito **RF-11**), y si la línea es visible, utiliza un controlador PID (Proporcional-Integral-Derivativo) enriquecido con predicción futura para calcular el ángulo exacto de dirección, cumpliendo el requisito **RF-06**. Simultáneamente, ajusta la velocidad de forma dinámica, frenando en las curvas cerradas para mantener la estabilidad, y orienta la propia cámara (servo "Pan") para "mirar" hacia la curva, consiguiendo de esta manera una precisión superior y evitando una posible pérdida de línea.

Capa 5: Aplicación

Esta capa es la encargada de conectar al cliente con el sistema. El módulo **webServer** actúa como el punto de integración de todo el sistema. Su función es inicializar y orquestar todos los módulos de hardware (servos, motores, luces), las funciones autónomas (**functions.py**) y la cámara (**camera_opencv.py**). Al arrancar, verifica el estado de la red Wi-Fi e intenta conectarse o crear su propio punto de acceso (AP), además de levantar dos servidores paralelos: uno HTTP con Flask (puerto 5000) dedicado exclusivamente a la transmisión del vídeo, y otro WebSocket asíncrono (puerto 8888) para el control en tiempo real.

El servidor WebSocket es el encargado de procesar todos los comandos que llegan desde la interfaz del usuario. Interpreta acciones manuales como mover el robot (**forward**, **left**), pero también activa modos autónomos como el seguimiento de línea (**trackLine**) o la búsqueda de colores, cumpliendo con el requisito **RF-09**. Cuando recibe estas órdenes, delega la ejecución a los módulos correspondientes: envía comandos a **move.py** para los motores, instrucciones a **RPIservo.py** para la dirección, o activa hilos en **cv_processor.py** y **functions.py** para la inteligencia artificial. Además, incluye un monitor de sistema ligero que lee directamente archivos del kernel de Linux (`/proc/stat`, `/sys/class/thermal`) para obtener datos de temperatura de CPU, uso de RAM y carga del procesador. Envía esta telemetría periódicamente al cliente web para mostrar el estado de salud del robot en el panel de control, todo ello gestionado de forma eficiente con `asyncio` para no bloquear el bucle de control principal.

La gestión del servidor Flask se delega en el módulo **app.py**. Su endpoint principal, *video_feed*, está diseñado para enviar un flujo continuo de imágenes ("MJPEG Stream") que cualquier navegador o cliente puede entender. Cuando alguien se conecta a esta ruta, **app.py** se conecta internamente a la clase **Camera** (que ya está corriendo gracias a **webServer.py**) y empieza a retransmitir frame tras frame en un bucle infinito, empaquetándolos en el formato estándar multipart/x-mixed-replace para que se vean como un vídeo fluido.

Por otro lado, tenemos la aplicación del lado del cliente **GUI.py**, este archivo constituye la interfaz de comando y control del cliente (como indica el requisito **RF-12**), desarrollada utilizando la biblioteca estándar **Tkinter** para proporcionar una plataforma robusta de teleoperación y monitorización en tiempo real. Actúa como el puente de interacción humano-máquina, estableciendo una comunicación bidireccional mediante dos protocolos distintos: un canal WebSocket asíncrono para la transmisión de comandos de control de baja latencia (dirección, tracción, gestión de modos autónomos) y recepción de telemetría del sistema (carga de CPU, temperatura, uso de memoria), y un canal HTTP dedicado a la recepción y renderizado del flujo de vídeo MJPEG proveniente del servidor Flask.

Adicionalmente, esta aplicación integra herramientas avanzadas de depuración para el subsistema de visión artificial. Al activar el modo de calibración (*FindColor*), la interfaz despliega dinámicamente controles deslizantes para el ajuste fino de los umbrales del espacio de color HSV. Esto permite al operador sintonizar los parámetros de segmentación visual en tiempo real sin necesidad de reiniciar el software del servidor, optimizando así el rendimiento de los algoritmos de seguimiento ante condiciones de iluminación variables.

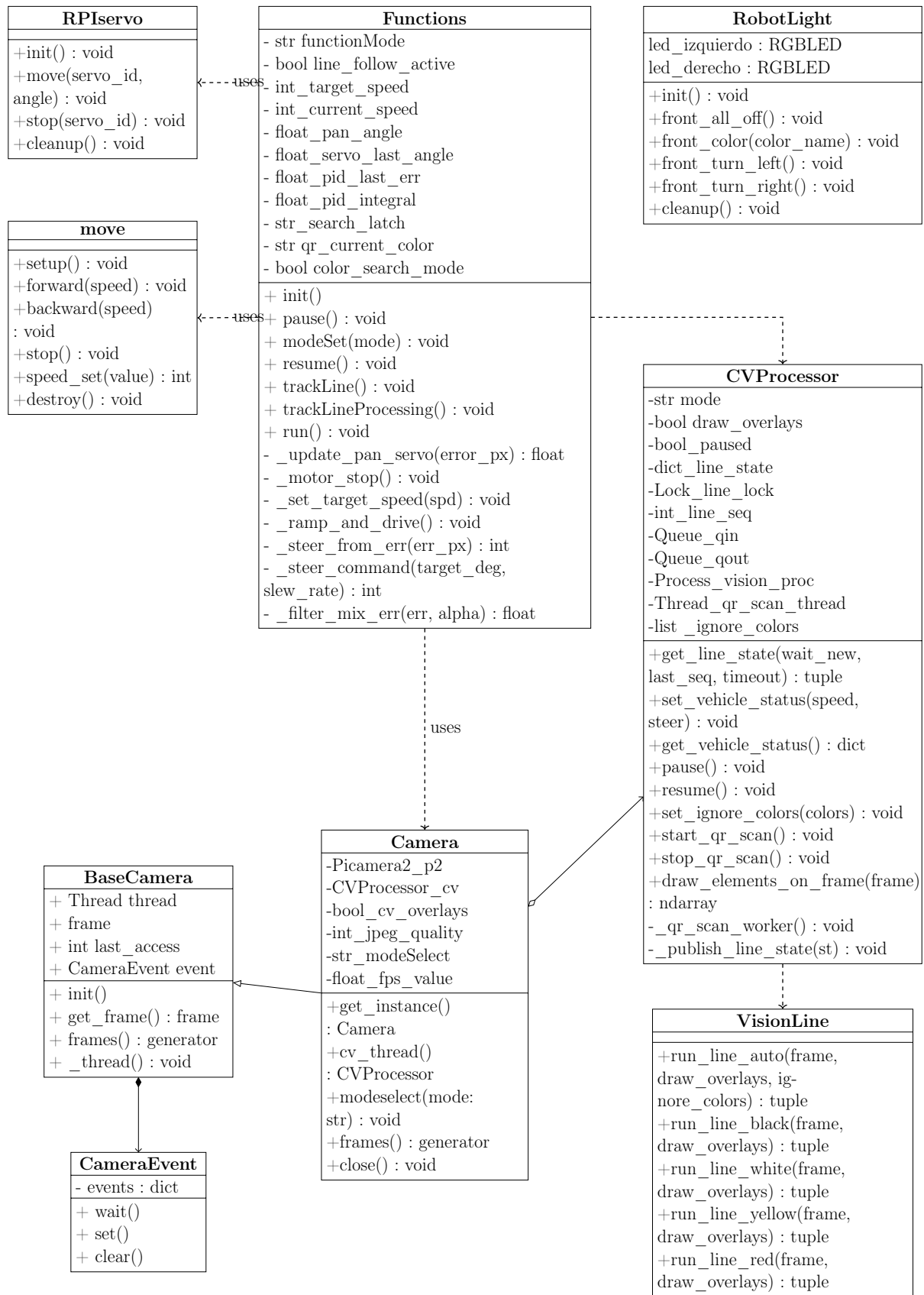


Figura 7: Diagrama de las Clases del Sistema.

Clase	Ref. Tabla	Descripción Funcional
Functions	<i>Tabla 9, 10 y 11</i>	Esta clase encapsula la inteligencia de navegación autónoma del robot. Se ejecuta en un hilo dedicado y actúa como el controlador principal que cierra el bucle entre la percepción y la acción: recibe continuamente los datos del análisis de visión (posición de la línea, curvatura) y aplica algoritmos de control (como PID y lógica de estados) para ajustar la dirección de los servos y la potencia de los motores. Maneja comportamientos complejos como la recuperación ante pérdida de línea, el control de velocidad adaptativo en curvas y la gestión de modos de operación.
CVProcessor	<i>Tabla 17 y 18</i>	Esta clase es el motor de procesamiento de visión artificial del proyecto. Para maximizar el rendimiento en la Raspberry Pi, utiliza multiprocesamiento (multiprocessing) para ejecutar los algoritmos de detección de líneas en un núcleo de CPU separado, evitando que el análisis intensivo bloquee el hilo principal del servidor. También gestiona un hilo secundario para la detección asíncrona de códigos QR y es responsable de dibujar la información de telemetría (HUD) y los gráficos de depuración sobre los frames de video antes de que sean mostrados al usuario.
Camera	<i>Tabla 15 y 16</i>	Esta clase implementa la lógica concreta de control del hardware de la cámara (usando Picamera2) y actúa como el productor principal de imágenes del sistema. Sigue el patrón Singleton para garantizar que solo exista una instancia controlando el dispositivo físico, evitando conflictos de hardware. Además de capturar imágenes raw, orquesta todo el flujo de procesamiento delegando en CVProcessor para el análisis de visión artificial y se encarga de codificar el resultado final en formato JPEG para su transmisión por red.
BaseCamera	<i>Tabla 14</i>	Esta clase abstracta establece los cimientos para el control de la cámara en el servidor. Su responsabilidad principal es gestionar el ciclo de vida del hilo de captura en segundo plano: inicia el hilo automáticamente cuando el primer cliente solicita video y lo mantiene vivo mientras sea necesario. Define la estructura que deben seguir las implementaciones concretas (como Camera), obligándolas a implementar el método generador de frames, y centraliza la lógica de acceso concurrente a la última imagen capturada.

Tabla 7: Resumen de las clases representadas en el diagrama UML de arquitectura.

Clase	Ref. Tabla	Descripción Funcional
CameraEvent	<i>Tabla 13</i>	Esta clase es una herramienta de sincronización multihilo diseñada para gestionar la distribución eficiente de frames de video a múltiples clientes. Utiliza primitivas de sincronización (<code>threading.Event</code>) para permitir que los hilos de los clientes esperen pasivamente hasta que haya una nueva imagen disponible, en lugar de consumir CPU constantemente comprobando el estado. Además, gestiona automáticamente el registro de clientes activos y la limpieza de los inactivos, asegurando que el sistema escale correctamente cuando hay varias conexiones simultáneas al stream de video.
RobotLight	<i>Tabla 12</i>	Esta clase proporciona una abstracción orientada a objetos para el subsistema de iluminación del robot. Oculta la complejidad de bajo nivel de la manipulación de pines GPIO, ofreciendo una interfaz sencilla para controlar los LEDs RGB delanteros. Sus métodos permiten establecer colores específicos, apagar las luces o invocar efectos predefinidos, facilitando la integración de señales visuales en la lógica del robot sin mezclar código de hardware en las capas superiores. Esta clase es utilizada desde el servidor (<code>webServer.py</code>) para controlar los LEDs del robot.

Tabla 8: Resumen de las clases restantes representadas en el diagrama UML de arquitectura.

Clase Functions - Atributos		
Nombre	Tipo	Descripción
Modos de Operación y Sincronización		
functionMode	str	Define el estado global del autómata ('none', 'auto', 'manual').
__flag	Event	Objeto <code>threading.Event</code> utilizado para pausar/reanudar hilos de forma segura.
_last_debug_log	float	Timestamp para controlar la frecuencia de impresión de logs y no saturar la consola.
Control de Tracción y Servos		
line_follow_active	bool	Flag maestro que habilita o deshabilita el bucle de seguimiento de línea.
_target_speed	int	Velocidad objetivo (PWM 0-100) que se desea alcanzar.
_current_speed	int	Velocidad actual aplicada a los motores (usada para rampas de aceleración).
_last_drive_time	float	Tiempo de la última orden enviada a los motores (para evitar saturación del bus I2C).
_pan_angle	float	Ángulo del servo de la cámara (Pan). Centrado por defecto en <code>PAN_CENTER</code> .
_servo_last_angle	int	Último ángulo enviado al servo de dirección.
_servo_last_time	float	Timestamp del último movimiento de dirección (para limitar la velocidad de giro).
Estado del Controlador PID		
_pid_last_err	float	Almacena el error del ciclo anterior (e_{t-1}) para calcular el término derivativo.
_pid_integral	float	Acumulador de errores para el término integral.
_pid_last_time	float	Tiempo exacto del último cálculo PID, vital para obtener un Δt real.
_err_ema	float	<i>Exponential Moving Average</i> . Filtro suavizado del error para reducir el ruido de la cámara.

Tabla 9: Relación de atributos de la clase `Functions`.

Clase Functions - Atributos		
Nombre	Tipo	Descripción
Lógica de Recuperación (Latch)		
_no_line_frames	int	Contador de fotogramas consecutivos sin detectar línea. Activa la parada de emergencia.
last_near_err	float	Último error válido detectado en la zona cercana. Usado para saber por dónde se perdió la línea.
last_near_side	int	Indica hacia qué lado estaba la línea antes de perderse (-1 Izq, 1 Der).
search_latch	str	Estado de "búsqueda bloqueada". Recuerda la dirección de giro necesaria para recuperar la línea.
search_debounce	int	Contador para evitar falsos positivos al decidir que se ha recuperado la línea.
_ignore_red_until	float	Temporizador para ignorar detecciones de líneas rojas tras un evento (debounce).
Gestión de Secuencias de Navegación		
white_sequence_index	int	Índice actual en la secuencia de acciones preprogramadas para líneas blancas.
yellow_sequence_index	int	Índice actual para la lógica de líneas amarillas (ej. reducción de velocidad).
target_color	str	Color específico que el robot debe buscar prioritariamente en el modo actual.
active_sequence	str	Nombre de la secuencia de navegación que se está ejecutando actualmente.
Sistema de Lectura QR		
qr_initial_color	str	Color de la línea antes de iniciar la maniobra QR.
qr_mode	str	Modo de navegación dictado por el QR (ej. "turn_left", "stop").
qr_cycles_total	int	Número de ciclos/pasos que debe durar la maniobra QR leída.
qr_cycles_done	int	Contador de ciclos ejecutados de la maniobra actual.
qr_needs_read	bool	Flag que indica si el sistema debe intentar leer un QR en el frame actual.

Tabla 10: Relación de atributos de la clase **Functions** (continuación).

Clase Functions		
Nombre	Tipo / Parámetros	Descripción
Atributos - Búsqueda de Color		
color_search_mode	bool	Indica si el robot está girando sobre su eje buscando un color específico.
color_search_target	str	El color objetivo que detendrá la búsqueda (ej. 'green').
color_search_direction	str	Dirección de giro forzada ('left' o 'right') durante la búsqueda.
color_search_frames	int	Contador de tiempo límite para abortar la búsqueda si no se encuentra nada.
Métodos		
__init__()	<i>Ninguno</i>	Constructor de la clase. Inicializa las variables de estado, establece los valores PID por defecto y prepara los objetos de hardware.
trackLine()	<i>Ninguno</i>	Método público de arranque. Configura el entorno, instancia el hilo de visión y activa el modo 'auto'.
pause()	<i>Ninguno</i>	Detiene los motores (PWM=0) y congela la lógica de control, pero mantiene los procesos en memoria listos para reanudar.
resume()	<i>Ninguno</i>	Restaura la tracción recuperando la <code>_target_speed</code> y reactiva el cálculo del PID tras una pausa.
modeSet(mode)	mode: str	Setter seguro para cambiar el modo de funcionamiento. Gestiona la lógica de transición para evitar estados inconsistentes.
trackLineProcessing()	<i>Ninguno</i>	Núcleo del sistema. Bucle infinito que orquesta la obtención de datos de visión, la ejecución del algoritmo PID y el envío de comandos a los actuadores.
PID_control(error)	error: float	Implementación matemática del algoritmo PID. Recibe el error visual y devuelve el ángulo de corrección necesario.
_steer_command(angle)	angle: float	Método de interfaz con la HAL. Traduce el ángulo lógico calculado por el PID a señales físicas para el servo, aplicando límites de seguridad (clamping).

Tabla 11: Descripción de los métodos de la clase **Functions**.

Clase RobotLight		
Nombre	Tipo / Parámetros	Descripción
Atributos de Hardware		
led_izquierdo	gpiozero.RGBLED	Objeto que representa el módulo LED físico situado en la parte frontal izquierda. Gestiona los pines GPIO 22 (R), 23 (G) y 24 (B) mediante PWM software.
led_derecho	gpiozero.RGBLED	Objeto que controla el módulo LED frontal derecho. Mapeado a los pines GPIO 10 (R), 9 (G) y 25 (B).
Métodos de Control		
__init__()	<i>Ninguno</i>	Constructor de la clase. Intenta inicializar la conexión con los pines GPIO a través de la librería gpiozero . Incluye manejo de excepciones para evitar el cierre del programa si el hardware falla.
front_color(name)	name: str	Establece el color de ambos faros delanteros buscando la cadena name (ej: 'red', 'cyan') en un mapa de colores interno. Realiza la inversión lógica necesaria para LEDs de ánodo común.
front_all_off()	<i>Ninguno</i>	Apaga completamente ambos módulos LED, poniendo a nivel bajo (o alto, según polaridad) todos los canales RGB para ahorrar energía.
front_turn_left()	<i>Ninguno</i>	Activa la señalización de giro a la izquierda. Actualmente configurado para encender luz blanca (alias de front_color), pero extensible para parpadeo ámbar.
front_turn_right()	<i>Ninguno</i>	Activa la señalización de giro a la derecha. Utilizado para indicar visualmente la intención del algoritmo de navegación en las bifurcaciones.

Tabla 12: Descripción de los atributos y métodos de la clase **RobotLight**.

Clase CameraEvent		
Nombre	Tipo / Parámetros	Descripción
Atributos		
events	dict	Diccionario que gestiona los suscriptores activos. Mapea el identificador único de cada hilo (<code>thread_id</code>) a una lista que contiene su objeto <code>threading.Event</code> y la marca de tiempo del último acceso.
Métodos de Sincronización		
<code>__init__()</code>	<i>Ninguno</i>	Constructor. Inicializa el diccionario de eventos vacío, preparando la estructura para registrar nuevos clientes HTTP.
<code>wait()</code>	<i>Ninguno</i>	Método de bloqueo. Si el cliente es nuevo, lo registra en el diccionario. Detiene la ejecución del hilo actual hasta que recibe la señal (<code>set</code>) de que hay un nuevo frame disponible.
<code>set()</code>	<i>Ninguno</i>	Notificador (Broadcaster). Despierta a todos los hilos que están esperando en <code>wait()</code> . Incluye lógica de <i>Garbage Collection</i> : elimina del diccionario a los clientes inactivos por más de 5 segundos.
<code>clear()</code>	<i>Ninguno</i>	Resetea la bandera del evento para el hilo actual, indicando que el frame ha sido consumido y debe esperar al siguiente ciclo de captura.

Tabla 13: Descripción de los atributos y métodos de la clase `CameraEvent`.

Clase BaseCamera		
Nombre	Tipo / Parámetros	Descripción
Atributos Estáticos (De Clase)		
thread	threading.Thread	Variable de clase compartida que almacena la referencia al hilo único de captura en segundo plano. Garantiza que no se instancien múltiples hilos de lectura de hardware.
frame	bytes	Buffer compartido que contiene el último fotograma capturado. Al ser estático, todos los clientes web leen de esta misma dirección de memoria.
last_access	float	Marca de tiempo (Timestamp) que registra cuándo fue la última vez que un cliente solicitó una imagen. Permite detener la cámara si no hay nadie mirando para ahorrar CPU.
event	CameraEvent	Instancia de la clase de eventos descrita anteriormente. Coordina la sincronización entre el hilo productor (Cámara) y los hilos consumidores (Flask).
Métodos		
__init__()	<i>Ninguno</i>	Constructor inteligente. Si el hilo de captura (thread) no existe, lo crea e inicia. Bloquea la ejecución inicial hasta que se obtiene el primer frame válido.
get_frame()	<i>Ninguno</i>	Método público principal. Actualiza el tiempo de acceso, espera la señal de sincronización y devuelve el fotograma actual (BaseCamera.frame) al cliente.
frames()	Generator	Método Abstracto. Define la interfaz que deben implementar las subclases (como Camera). Debe devolver un iterador que genere fotogramas continuamente. Lanza error si no se sobrescribe.
_thread()	@classmethod	Bucle principal del hilo en segundo plano (Worker). Itera sobre el generador frames() , actualiza el buffer global y dispara el evento set() para notificar a los clientes.

Tabla 14: Descripción de los atributos y métodos de la clase abstracta **BaseCamera**.

Clase Camera		
Nombre	Tipo / Parámetros	Descripción
Atributos		
<code>_instance</code>	<code>Camera</code>	Variable estática que almacena la única instancia permitida de la clase. Fundamental para la implementación del patrón <i>Singleton</i> , garantizando que múltiples hilos no intenten acceder al puerto de cámara simultáneamente.
<code>_p2</code>	<code>Picamera2</code>	Objeto controlador de bajo nivel de la librería <code>libcamera</code> . Gestiona la configuración del sensor, el balance de blancos y la captura directa de arrays en memoria RAM.
<code>_cv</code>	<code>CVProcessor</code>	Referencia estática al procesador de visión artificial. Se utiliza para inyectar el fotograma crudo en el algoritmo de detección antes de comprimirlo para la web.
<code>_jpeg_quality</code>	<code>int</code>	Parámetro de configuración (0-100) que define el nivel de compresión de la imagen de salida. Se ajusta para equilibrar la calidad visual con el ancho de banda de la red Wi-Fi.
<code>_fps_value</code>	<code>float</code>	Métrica de rendimiento calculada en tiempo real. Almacena la tasa de cuadros por segundo (FPS) actual del pipeline de captura y procesamiento.

Tabla 15: Descripción de los atributos de la clase **Camera**.

Clase Camera		
Nombre	Tipo / Parámetros	Descripción
Métodos		
<code>__new__()</code>	<i>Ninguno</i>	Método mágico de instanciación. Intercepta la creación del objeto para devolver siempre la instancia existente en <code>_instance</code> o crear una nueva si no existe (Singleton).
<code>__init__()</code>	<i>Ninguno</i>	Configura el hardware de la cámara. Establece la resolución (640x480), el formato de color (RGB888) y los parámetros de ajuste automático (tuning) de Picamera2.
<code>set_cv_processor(cv)</code>	<code>cv: CVProcessor</code>	Método estático de inyección de dependencias. Vincula el objeto de procesamiento de visión con la cámara, evitando importaciones circulares entre módulos.
<code>frames()</code>	Generator	Pipeline principal. Sobrescribe el método abstracto de la clase base. En un bucle infinito: 1) Captura el array crudo, 2) Lo envía a <code>_cv</code> para análisis, 3) Superpone datos de telemetría (HUD), 4) Comprime a JPEG y 5) Entrega el stream a la web.

Tabla 16: Descripción de los métodos de la clase **Camera**.

Clase CVProcessor		
Nombre	Tipo / Parámetros	Descripción
Atributos de Concurrencia (IPC)		
<code>_qin</code>	<code>mp.Queue</code>	Cola de entrada (Input Queue) segura para hilos. Actúa como buffer donde el hilo principal deposita los frames crudos para que sean consumidos por el proceso hijo de visión.
<code>_qout</code>	<code>mp.Queue</code>	Cola de salida (Output Queue). Canal por donde el proceso hijo devuelve los resultados del análisis (diccionario de estado e imagen procesada) al hilo principal.
<code>_vision_proc</code>	<code>mp.Process</code>	Objeto que encapsula un proceso del sistema operativo independiente. Ejecuta el bucle infinito de visión artificial en un núcleo de CPU separado para evitar el bloqueo del GIL (<i>Global Interpreter Lock</i>).
<code>_line_state</code>	<code>dict</code>	Diccionario protegido por cerrojos (<i>mutex</i>). Actúa como la "memoria a corto plazo" del robot, almacenando la última posición conocida de la línea, el error y los datos QR decodificados.

Tabla 17: Descripción de los atributos de la clase `CVProcessor`.

Clase CVProcessor		
Nombre	Tipo / Parámetros	Descripción
Métodos		
<code>__init__()</code>	<i>Ninguno</i>	Constructor. Inicializa las colas de comunicación, arranca el subproceso <code>_vision_proc</code> y prepara el hilo secundario para la lectura de códigos QR.
<code>draw_elements_on_frame(frame)</code>	<code>frame: numpy.ndarray</code>	Método Orquestador. Recibe la imagen de la cámara, la introduce en <code>_qin</code> y recupera (si existe) el resultado más reciente de <code>_qout</code> . Si el proceso va lento, descarta frames antiguos.
<code>get_line_state()</code>	<code>dict</code>	Getter <i>thread-safe</i> . Devuelve una copia del estado actual de la visión. Es utilizado por la clase <code>Functions</code> para alimentar el algoritmo PID.
<code>start_qr_scan()</code>	<i>Ninguno</i>	Activa el hilo de escaneo QR utilizando <code>pyzbar</code> en paralelo al seguimiento de línea.
<code>stop_qr_scan()</code>	<i>Ninguno</i>	Detiene el hilo de escaneo QR.
<code>set_ignore_colors(colors)</code>	<code>colors: list</code>	Define qué colores (ej. 'yellow', 'white') deben ser ignorados por el algoritmo de visión.
<code>pause()</code> / <code>resume()</code>	<i>Ninguno</i>	Controlan la pausa y reanudación del procesamiento de visión para gestionar el consumo de CPU.
<code>set_vehicle_status(v, s)</code>	<code>v: int, s: int</code>	Actualiza los datos de telemetría para el HUD.
<code>_vision_worker_main()</code>	Static Function	Función aislada (ejecutada en proceso hijo) que contiene el bucle infinito de procesamiento matemático.

Tabla 18: Descripción de los métodos de la clase `CVProcessor`.

5.2.2. Modelo Procedimental

Una vez establecidos y aclarados todos los componentes software del sistema, es momento de explicar como funciona el sistema en su conjunto. Para ello, se va a abordar todo el proceso desde el inicio de la ejecución hasta el fin de un ciclo de ejecución.

La figura 8 muestra la dinámica de ejecución del sistema.

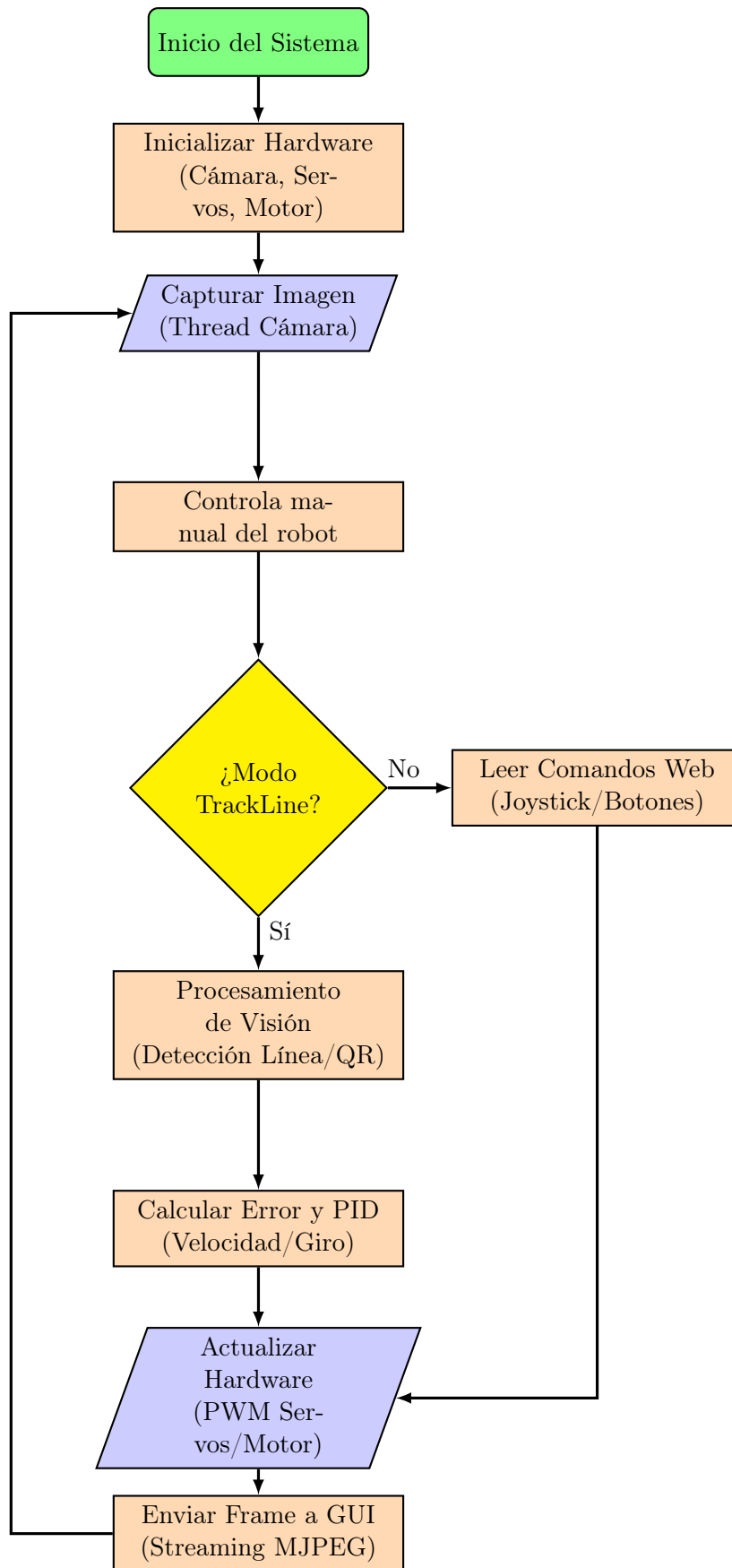


Figura 8: Diagrama de flujo general del robot.

Como se puede observar en la figura 8, el sistema comienza con la inicialización del hardware, que incluye la cámara, los servos y el motor. Una vez inicializado el hardware, el sistema entra en un bucle principal que se ejecuta de forma continua. En cada iteración del bucle, el sistema captura una imagen de la cámara. Dependiendo del modo de control, el sistema decide si procesar la imagen para detectar la línea y los códigos QR o si comprimirla, enviarla al cliente y leer los comandos del joystick y los botones enviados por el cliente. En el caso de estar en el modo **TrackLine**, el sistema calcula el error y el PID, que se utilizan para actualizar el hardware y enviar el frame a la GUI. En el caso de estar en el modo **Manual**, el sistema envía el frame a la GUI y espera los comandos del joystick y los botones.

Tras describir el funcionamiento general del sistema, vamos a pasar a explicar el funcionamiento de las diferentes partes que lo componen basándonos en los módulos explicados en la sección anterior.

5.2.3. Flujo de visión

Para la obtención de la imagen, el sistema utiliza la cámara del robot. La imagen se captura en gracias a la clase **Camera**. Dependiendo de si está activado el modo **TrackLine**, el sistema comprime la imagen para enviarla al cliente en caso de que no esté activado. Si el modo **TrackLine** está activado, el sistema envía la imagen a la clase **CVProcessor** para que se procese en el módulo **vision_line** y se envíen los overlays al cliente y el diccionario state a la clase **Function**.

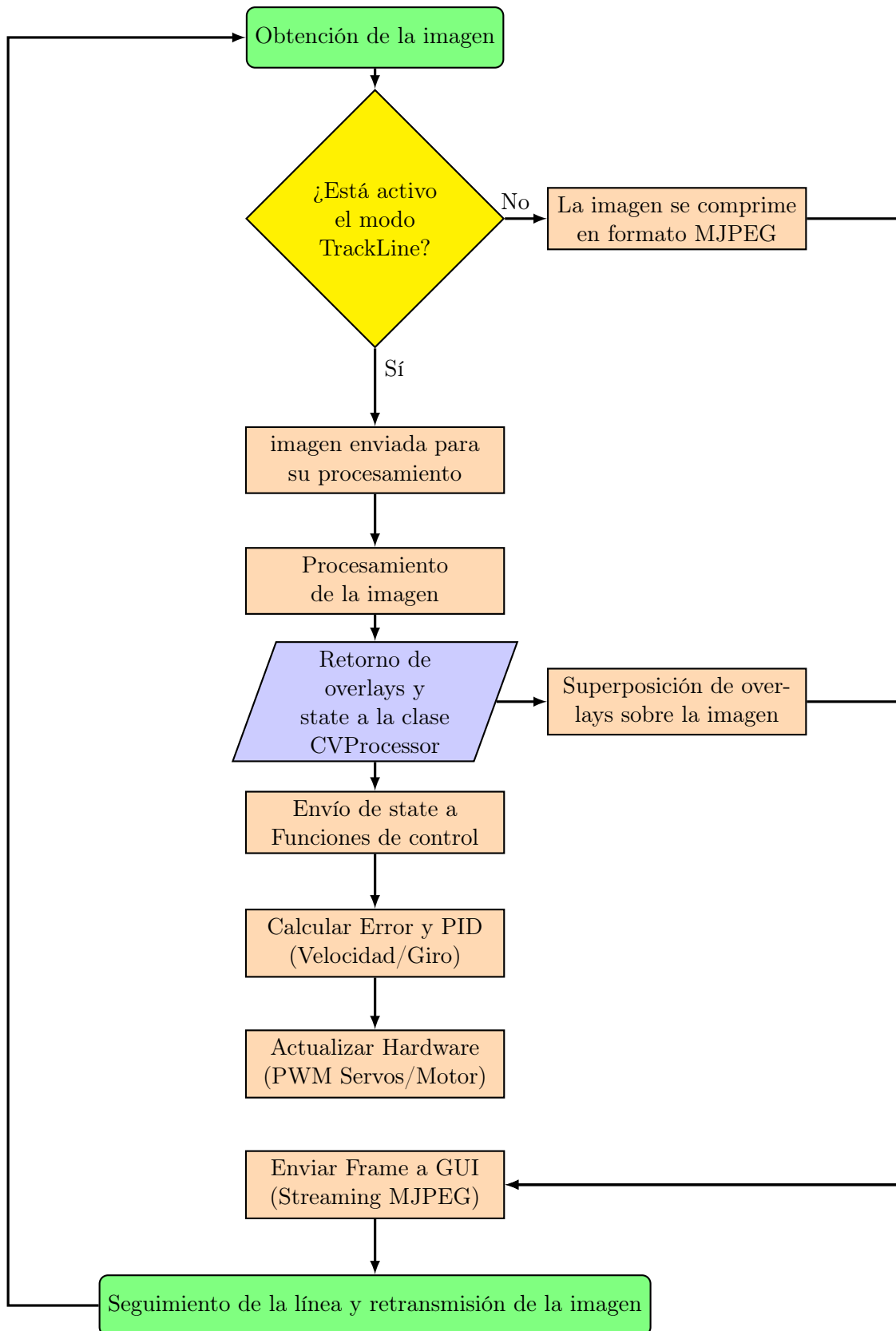


Figura 9: Diagrama de flujo de la visión del robot.

5.2.4. Flujo de Control

El flujo de control abarca desde la obtención de los datos de la imagen que se puede ver en la figura 9, hasta el seguimiento de la línea y las posibles acciones que se pueden realizar.

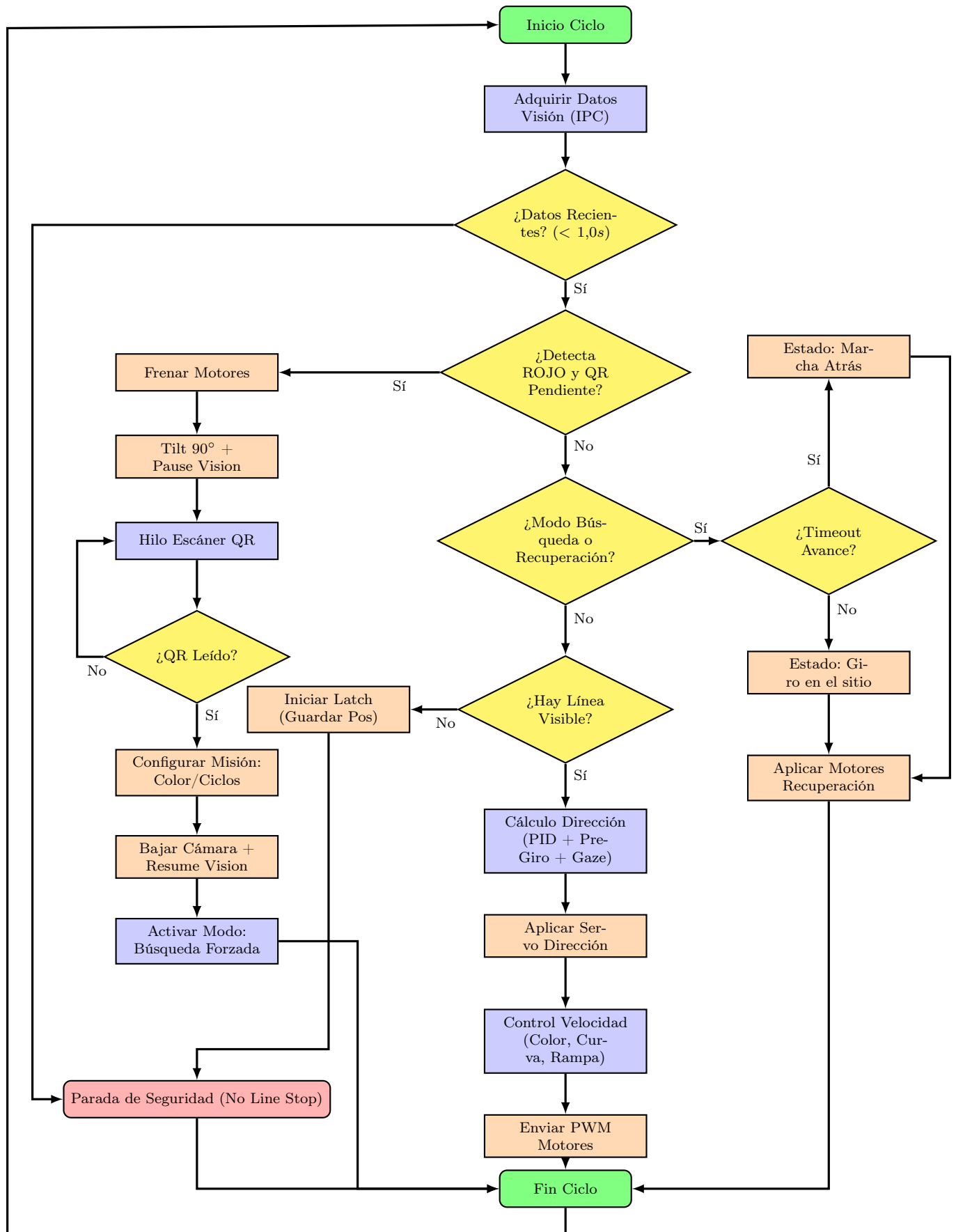


Figura 10: Diagrama de flujo de la lógica de control del robot.

Tras adquirir los datos de la imagen, el sistema verifica la integridad temporal de la información recibida. Si los datos de visión tienen una antigüedad superior a un umbral predefinido (generalmente un segundo), el algoritmo asume un fallo en la captación o transmisión de imágenes. En este escenario, el flujo se deriva inmediatamente hacia una rutina de parada de seguridad que corta la potencia de los motores, previniendo así cualquier comportamiento errático que pudiera surgir de navegar con información obsoleta.

Una vez validada la fiabilidad de los datos, el algoritmo evalúa la existencia de eventos prioritarios que deban interrumpir la navegación estándar, siendo el más relevante la detección de una línea roja transversal asociada a códigos QR. Si el sistema identifica esta señal y el vehículo tiene una lectura pendiente, se suspende temporalmente el seguimiento de línea para iniciar una secuencia de eventos específica. El robot detiene sus motores y reorienta la cámara mecánicamente para enfocar el código QR; tras decodificar la nueva misión (que puede implicar cambios en el color de línea a seguir o en el contador de vueltas), el sistema reanuda la visión y fuerza un estado de búsqueda del nuevo objetivo, ignorando la línea de parada original para continuar el circuito.

En ausencia de eventos de interrupción, el núcleo de la decisión de movimiento depende de la visibilidad de la línea en el instante actual. Si el sensor no detecta el carril, se activa un modo de recuperación escalonado lógico de "Latch". Este mecanismo permite al vehículo mantener brevemente su última trayectoria conocida para superar pequeños huecos o fallos momentáneos en la detección. Sin embargo, si la pérdida de línea persiste más allá de un tiempo de seguridad, el sistema transiciona a maniobras de reorientación activa, ejecutando giros sobre el eje o movimientos de marcha atrás diseñados para volver a situar el carril dentro del campo de visión de la cámara.

Finalmente, cuando la línea es visible, se ejecuta el control de navegación nominal que coordina tanto la dirección como la velocidad. Para la dirección, se emplea un controlador PID que ajusta el ángulo de las ruedas basándose en el error de posición lateral, complementado por una predicción de curvatura (FeedForward) y un ajuste activo de la cámara (Active Gaze) que anticipa el trazado. Simultáneamente, el control de velocidad modula la potencia de los motores en función del contexto: se establecen límites máximos según el color de la línea detectada y se aplica un frenado dinámico proporcional a la severidad de la curva, asegurando que el vehículo reduzca su velocidad automáticamente antes de afrontar giros cerrados para garantizar la estabilidad.

Conexión Cliente-Servidor

Tras abordar el flujo de visión y el flujo de control, solo queda hablar de la conexión cliente-servidor para conocer todo el funcionamiento del sistema.

El proceso de conexión comienza en la Raspberry Pi con la ejecución del script **webServer.py**. Antes de aceptar cualquier conexión externa, el servidor realiza una secuencia de *bootstrapping* o arranque crítico: primero inicializa los controladores de bajo nivel para los motores y servos (a través de GPIO e I2C) y levanta la instancia de la cámara. Una vez que el hardware está listo, el sistema despliega dos servicios de red paralelos. Por un lado, inicia un servidor HTTP ligero en el puerto 5000 (usando Flask) dedicado exclusivamente a la transmisión de vídeo. Por otro lado, levanta un servidor de WebSockets asíncrono en el puerto 8888, que actuará como el canal de control en tiempo real para recibir comandos y enviar telemetría.

Desde el lado del cliente (el ordenador), la aplicación **GUI.py** al ser ejecutada genera una ventana con la interfaz del cliente. La aplicación permanece en espera hasta que se introduce la dirección IP del robot en la interfaz. Al pulsar *Connect*, el cliente inicia un handshake TCP contra el puerto 8888 del servidor. Para garantizar la seguridad del control, el sistema implementa un protocolo de autenticación simple: el cliente envía inmediatamente una cadena de credenciales predefinida (admin:123456). El servidor verifica este mensaje y, solo si es correcto, responde con la confirmación *congratulation*. Si este intercambio falla, la conexión se cierra inmediatamente, impidiendo el acceso no autorizado a los controles del vehículo.

Una vez autenticado el canal de control, el cliente inicia un segundo hilo de ejecución independiente para gestionar la visión. Este hilo realiza una petición HTTP GET al puerto 5000 del servidor (/video_feed). El servidor Flask responde con un flujo continuo de datos en formato MJPEG (Motion JPEG), enviando fotograma tras fotograma en un bucle infinito. El cliente recibe estos bytes, los decodifica y actualiza la ventana de la interfaz gráfica en tiempo real. Al separar el vídeo (HTTP) del control (WebSocket), se evita que la transmisión de imágenes pesadas sature o retrase los comandos críticos de movimiento.

Con ambos canales (vídeo y control) activos, el sistema entra en su fase operativa. El cliente mantiene el canal WebSocket abierto permanentemente para enviar los comandos del usuario (como *forward* o *trackLine*) a través de la interfaz del cliente, con una latencia mínima. Paralelamente, inicia un ciclo de interrogación automático: cada 2 segundos solicita al servidor el estado de salud del sistema (get_info). El servidor responde leyendo sus sensores internos y devolviendo un paquete JSON con la temperatura de la CPU, el uso de RAM y la carga del procesador, permitiendo al operador monitorizar la *salud* del robot mientras lo conduce.

El control manual del robot por parte del usuario se da por medio de los botones de la interfaz o las teclas WASD del teclado, que permiten elegir la dirección en la que debe avanzar el robot manualmente. En la interfaz del cliente también se encuentra otra serie de botones, que permiten controlar el movimiento de la cámara y activar y desactivar módulos como el de seguimiento de línea (*trackLine*). De esta forma se cumple con el requisito funcional **RF-12** visto en la Tabla 2.

En la figura 11 se puede observar el conjunto de procesos explicados:

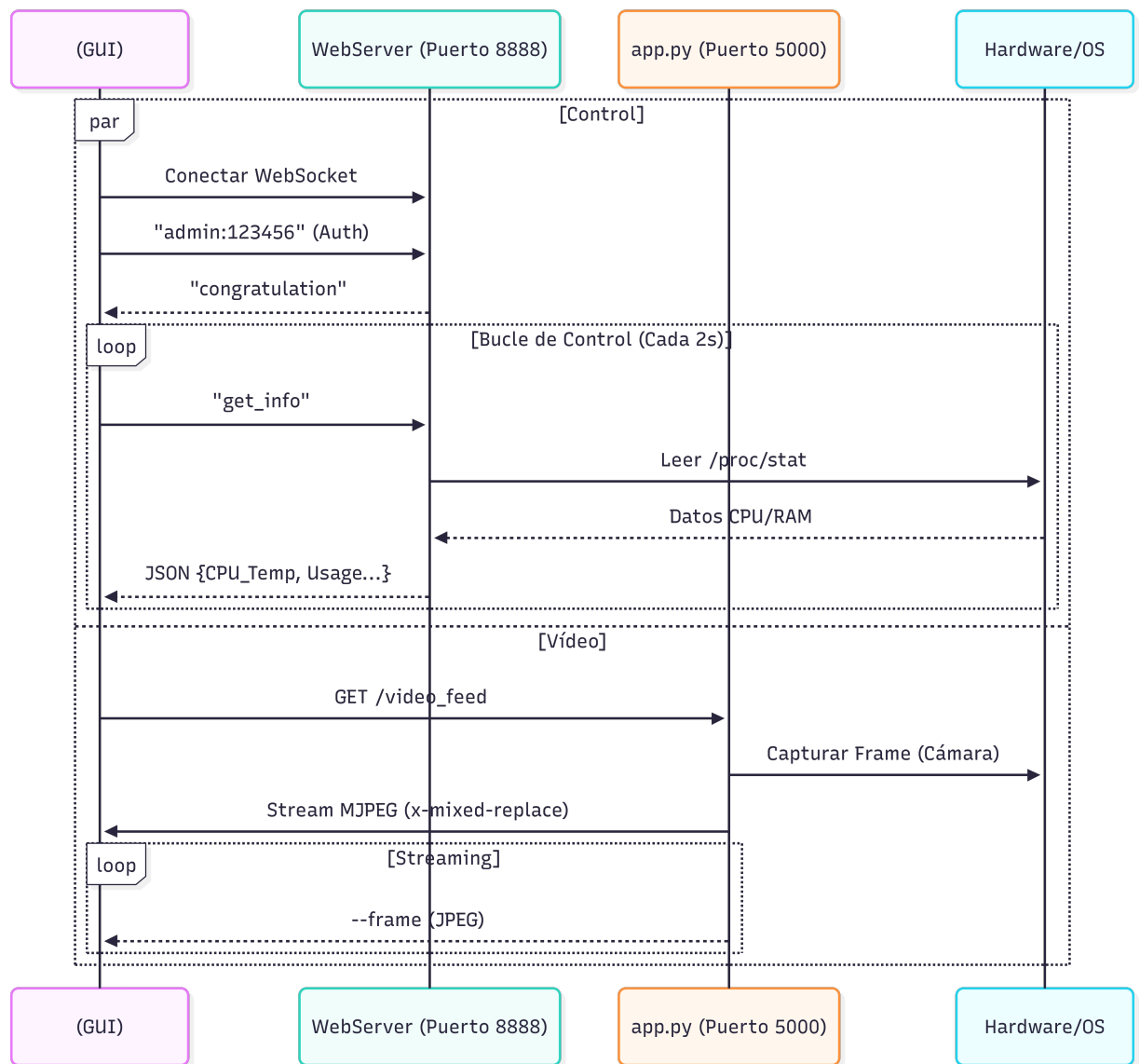


Figura 11: Diagrama de secuencia: Protocolo de conexión y comunicación entre el cliente y el servidor.

Capítulo 6

Pruebas

Este capítulo está dedicado a detallar las pruebas que el prototipo ha enfrentado y superado, demostrando su capacidad para navegar de forma autónoma y segura. El objetivo es demostrar la evolución del sistema y cómo se ha ido mejorando a lo largo del desarrollo del proyecto.

Las pruebas que se van a detallar a continuación se realizaron sobre diferentes circuitos, cada uno con diferentes características. En cada circuito se hicieron varias pruebas y gracias a sus resultados se implementaron mejoras en el sistema.

6.1. Prueba 1: Conducción Autónoma con Sensor Infrarrojo

Tras el montaje y la implementación de la lógica de control, el robot está listo para someterse a una primera prueba de conducción autónoma. Para ello se ha diseñado un circuito simple que sea capaz de mostrar las capacidades y carencias del robot en esta fase inicial del proyecto.

6.1.1. Descripción de los Componentes Utilizados

Esta versión del sistema lleva incorporado un sensor infrarrojo para la detección de la línea, dicho sensor lo lleva incorporado debajo del eje de las ruedas delanteras, de tal forma que pueda detectar si la línea está a la izquierda, derecha o en el centro cuando pase por encima de ella. El sistema procura mantener la línea en el centro del robot, en el caso de que se detecte en alguno de los laterales, el sistema corregirá la dirección haciendo uso del servo que controla las ruedas.

Aparte de la incorporación del sensor infrarrojo, el resto de componentes son los detallados en el capítulo anterior de este documento. En la figura 12 se puede ver una imagen de esta versión del robot.

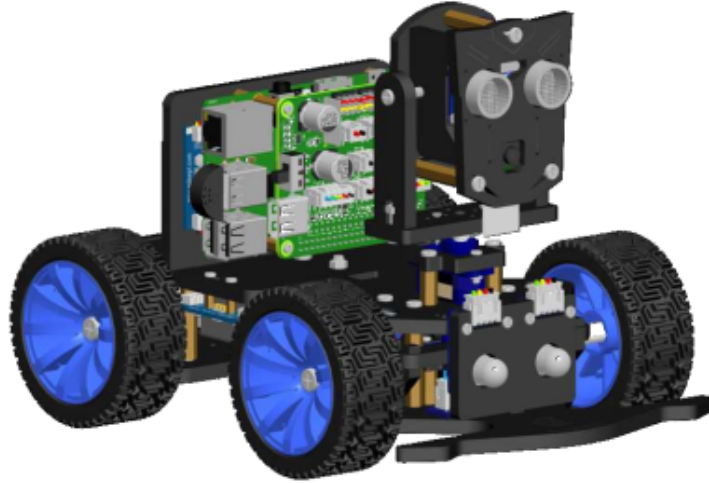


Figura 12: Vista del robot.

En la figura 13 se puede observar donde va ensamblado el sensor infrarrojo.

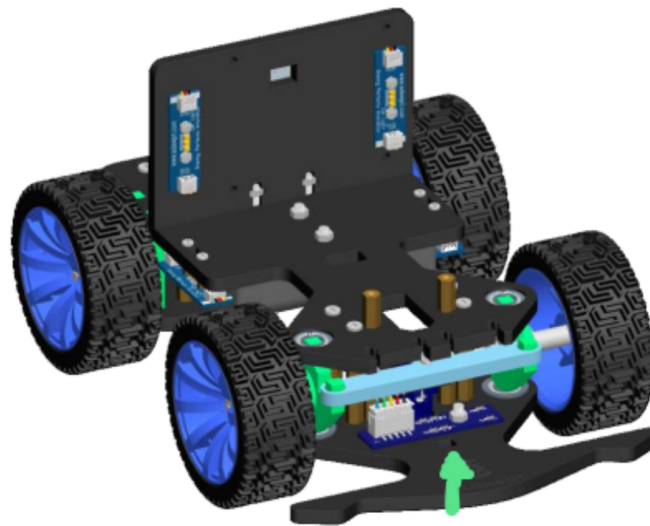


Figura 13: Vista del sensor infrarrojo.

6.1.2. Funcionamiento del Sistema

El funcionamiento del sistema consiste en que sea capaz de detectar y avanzar siguiendo la línea negra hasta una bifurcación, donde deberá pararse y hacer un barrido moviendo la cámara en busca de un código QR que le indique por donde debe avanzar. Tras leer y comprender el significado de las indicaciones, el robot debe reanudar la marcha siguiendo la línea negra por donde se le ha indicado.

Para que la prueba sea un éxito, el robot deberá cumplir con lo anterior sin salirse de la línea. También deberá leer el QR y reaccionar de forma correcta. Estas directrices son recogidas en la tabla 19, donde se han traducido a los requisitos que se deben cumplir.

6.1.3. Circuito Desarrollado

En la figura 14 se observa el circuito diseñado para la prueba. Al ser una fase temprana del desarrollo, se ha optado por un circuito sencillo, con una recta, una curva en la que se ponga a prueba su precisión a la hora de seguir la línea y una bifurcación. La bifurcación están diseñadas con una línea en perpendicular a la línea por la que circula el robot, esto ocurre porque el robot detecta una bifurcación cuando su sensor infrarrojo ve la línea tanto por la izquierda, como por la derecha y por el centro. Es de esta forma que el robot sabe que está en una bifurcación y por tanto debe buscar en sus alrededores un código QR.

Se ha añadido una pequeña línea perpendicular a la línea negra en uno de los extremos para marcar el punto de inicio del circuito y que quede más en la imagen de la figura 14.



Figura 14: Vista del Primer Circuito.

6.1.4. Requisitos a Cumplir

Al tratarse de una primera prueba, los objetivos de la misma son más simples, centrados en la conducción autónoma de un robot sigue-líneas tradicional. En la tabla 19 se recogen los requisitos que la prueba pretende evaluar de la conducción del robot.

Tabla 19: Requisitos funcionales (RF) de la Primera Prueba.

Código	Descripción
RF01	El sistema debe ser capaz de detectar la línea negra gracias al sensor infrarrojo que lleva incorporado.
RF02	El sistema debe procesar los datos obtenidos por el sensor infrarrojo (RF-01) en tiempo real, de tal forma que pueda adaptarse al trazado de la línea sin perderla.
RF03	El sistema debe ser capaz de detectar la presencia de códigos QR en el entorno con el uso de la cámara.
RF04	El sistema debe ser capaz de detectar cuando se encuentra en una bifurcación y actuar en consecuencia.
RF05	El sistema debe ser capaz de interpretar las instrucciones proporcionadas por el QR leído (RF-03).

6.1.5. Resultados Obtenidos

Tras poner a prueba el robot en el circuito se han sacado las siguientes conclusiones:

- **Seguimiento de línea:** El robot ha presentado dificultades a la hora de seguir la línea, principalmente en curvas. Tras pequeñas modificaciones en el código se ha detectado que el fenómeno ocurre por falta de precisión en el sensor infrarrojo, haciendo que a veces el vehículo pase por encima de la línea sin ser detectada. Esto provoca que el vehículo pierda la línea y quede a la deriva sin ningún tipo de referencia, dado que en la lógica de control no se ha planteado un escenario de pérdida de línea total. Es por ello que el sistema no ha conseguido cumplir totalmente con el requisito funcional **RF-01** de la tabla 19.
- **Detección en bifurcaciones:** Uno de los puntos clave de la prueba era observar su comportamiento en las bifurcaciones. Si bien, se ha desenvuelto bien a la hora de detectar el QR e interpretar las instrucciones de este, cumpliendo por tanto los requisitos **RF-03** y **RF-05**. Al igual que ha ocurrido con el seguimiento de línea, en ciertas ocasiones, el robot ha presentado dificultades para detectar la bifurcación a causa del sensor infrarrojo, es por tanto que no se puede interpretar que haya cumplido con el requisito **RF-04** al completo.

6.1.6. Conclusiones de la Prueba 1

Tras describir los componentes del robot y del circuito para esta prueba y analizar los resultados obtenidos, se puede decir que el robot no ha superado por completo la prueba.

Al observar los datos que ha presentado el robot, se llega a la conclusión de que los principales problemas han sido debido a la detección de la línea por parte del sensor infrarrojo, que no cuenta con la precisión o tiempo de respuesta suficiente para este proyecto.

Por otro lado, el uso del sensor infrarrojo dificulta el cumplimiento de algunos requisitos del proyecto, como el requisito **RF-04** de la Tabla 2, que trata sobre la percepción cromática. Es por ello, que se ha optado por basar la lógica del seguimiento de línea a la cámara del robot, de tal forma que se pueda detectar el color de la línea y que el sistema reaccione ante este. De esta forma se dotaría al sistema de una mayor adaptabilidad al entorno, abriendo al mismo tiempo el abanico de opciones que el sistema puede ofrecer.

6.2. Prueba 2: Conducción Autónoma con Visión Artificial

Después de los resultados obtenidos en la primera prueba, el trabajo pasa a centrarse en la visión artificial, ya que el seguimiento de la línea dependería de ella.

6.2.1. Descripción de los Componentes Utilizados

En esta ocasión se prescindió del sensor infrarrojo, puesto que no iba a ser necesario si todo el seguimiento de la línea iba a ser a través de la visión de la cámara. Por eso mismo se decidió sacar la cámara de su carcasa y colocarla un poco más arriba, para que la visión abarcara toda la superficie posible.

El robot quedaría con los componentes descritos en la Tabla 5, en la imagen se puede observar la que sería la segunda versión del robot.

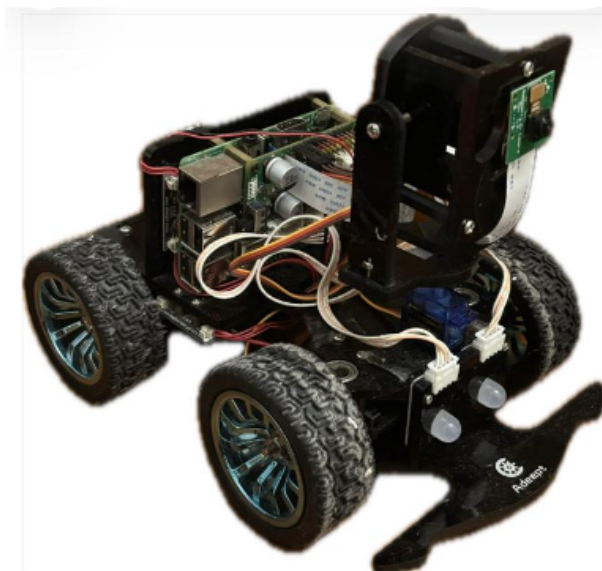


Figura 15: Segunda Versión del Robot

6.2.2. Funcionamiento del Sistema

En este caso el funcionamiento del sistema no consistirá en seguir una línea de un solo color simplemente, sino que el robot tendrá que seguir la línea por un circuito compuesto por rectas y curvas pronunciadas de diferentes colores. De esta manera demostrará que es capaz de percibir correctamente los colores de cada línea y que no tendrá problemas en que la línea cambie de color.

Al ser varias curvas y más pronunciadas que en el circuito anterior, el robot deberá adaptar su velocidad para no salirse de la línea.

En la prueba anterior se vio como el robot seguía adelante o se quedaba quieto si no percibía la línea. Este comportamiento limita la capacidad del robot, puesto que en el caso de que pierda la línea de vista el robot no tendrá forma de reincorporarse al circuito. Es por ello que se ha optado por implementar un sistema de seguimiento de línea que permita al robot reaccionar ante la pérdida de la línea.

Para que la prueba sea un éxito, el robot deberá cumplir con lo que se acaba de mencionar. En esta prueba no será necesario que el robot lea ningún código QR, puesto que se pretende evaluar la capacidad de detección de la línea de diferentes colores y la capacidad de adaptarse al circuito. Estas directrices son recogidas en la tabla 20, donde se han traducido a los requisitos que se deben cumplir.

6.2.3. Circuito Desarrollado

En la figura 16 se observa el circuito diseñado para la prueba. Al tratarse en este caso de una segunda prueba, el circuito ha crecido en complejidad y longitud, lo que permitirá evaluar la capacidad del robot para adaptarse a diferentes situaciones.

Para poner a prueba las capacidades del robot en este caso, se ha optado por hacer un circuito que combine rectas y curvas pronunciadas de diferentes colores, como hemos mencionado anteriormente. De esta manera se evaluará la capacidad del robot para adaptarse a diferentes situaciones y la capacidad de detección de la línea de diferentes colores. Es por ello también, que en este circuito no se ha implementado ninguna bifurcación, ni lectura de códigos QR, puesto que ya fueron evaluadas en caso de la primera prueba y los objetivos de esta pruebas son otros.

Las curvas se han hecho más pronunciadas en este caso, con el fin de ver la respuesta del robot ante la posible pérdida de línea y la capacidad de adaptarse al circuito. En la curva más pronunciada, el robot deberá reducir la velocidad para no salirse de la línea y hacer uso de una maniobra de marcha atrás que le permita reposicionarse para continuar con el circuito.

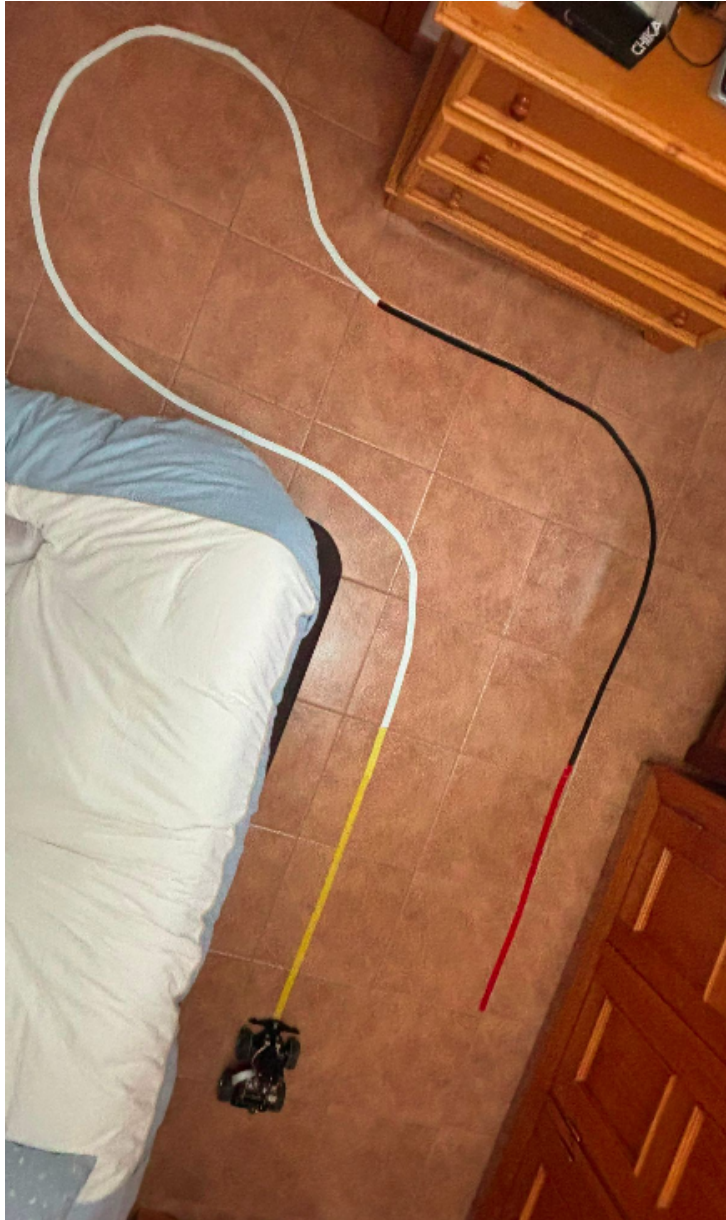


Figura 16: Vista del Segundo Circuito.

6.2.4. Requisitos a Cumplir

Teniendo ya experiencia de la primera prueba, los objetivos en este caso son más algo más complejos, centrados en la conducción autónoma de un robot sigue-líneas con visión artificial. En la tabla 20 se recogen los requisitos que la prueba pretende evaluar de la conducción del robot.

Tabla 20: Requisitos funcionales (RF) de la Segunda Prueba.

Código	Descripción
RF01	El sistema debe ser capaz de detectar líneas de colores amarillo, negro, blanco y rojo.
RF02	El sistema debe ser capaz de seguir las líneas detectadas en el requisito RF-01 en tiempo real e independientemente del color que tengan (siempre y cuando sea alguno de los colores mencionados en el requisito RF-01).
RF03	El sistema debe ser capaz de detectar cuando se encuentra en una curva y adaptar su velocidad.
RF04	En caso de pérdida de línea, el sistema debe girar en la dirección donde se vio la línea por ultima vez, con el fin de recuperar la línea .
RF05	En caso de que al producirse el requisito RF-04 no se recupere la línea, el sistema debe girar en la dirección opuesta a la que se vio la línea por ultima vez y dar marcha atrás, con el fin de reposicionarse y recuperar la línea .

6.2.5. Resultados Obtenidos

Tras poner a prueba el robot en el circuito se han sacado las siguientes conclusiones:

- **Seguimiento de línea:** El robot, tras un ajuste de los colores de la línea, ha sido capaz de seguir la línea en la mayoría del recorrido, independientemente del color que tenga, cumpliendo por tanto con el requisito **RF-01** y **RF-02**. Por otra parte el el robot ha perdido la línea en algunas curvas, lo que ha permitido ver su capacidad de recuperación, si bien en algunas ocasiones le ha supuesto mayor problema recuperar la línea, se ha observado que la maniobra de marcha atrás suele ser bastante efectiva, por lo que se puede decir que ha cumplido con el requisito **RF-04** y **RF-05**.
- **Regulación de velocidad:** Tras ver que el robot ha sido capaz de seguir el circuito sin grandes problemas, se ha puesto especial atención en las curvas del circuito, donde el vehículo debía reducir su velocidad para no salirse de la línea. En este sentido, se ha observado que el robot es capaz de reducir su velocidad en las curvas y adaptarse a la forma de estas, cumpliendo por tanto con el requisito **RF-03**.

6.2.6. Conclusiones de la Prueba 2

Tras el cambio en algunos los componentes del robot, el cambio radical en la lógica de control, el diseño del circuito para esta prueba y haber analizado los resultados obtenidos, se puede decir que el robot ha superado sin mayores complicaciones esta prueba.

Si bien a la hora de hacer la prueba ha habido que ajustar varios parámetros del código, una vez encontrados los valores óptimos, el robot ha sido capaz de seguir la línea de forma estable y autónoma.

Por otro lado, algunos problemas que no se tenían contemplados en un inicio han surgido durante las pruebas, como por ejemplo el gran efecto que tiene la luz a la hora de percibir los colores por la cámara. La iluminación del entorno supone un problema que hay que

gestionar a la hora de establecer los parámetros de los colores. La percepción de estos colores puede verse significativamente alterada si la iluminación cambia drásticamente.

Otro problema que ha surgido durante la elaboración de la lógica de control ha sido el de la alta latencia que tenía el sistema en un inicio. Esto ocurre debido a que había un solo proceso a la hora de obtener la imagen de la cámara y procesarla. Para solucionar este problema, se ha hecho uso de múltiples hilos de ejecución, de tal forma que se pueda obtener la imagen de la cámara y procesarla en tiempo real. De esta forma se reduce la latencia del sistema y se mejora la estabilidad del seguimiento de línea.

Finalmente, tras haber realizado esta prueba exitosamente, se trabajará en solucionar el principal problema que ha surgido al implementar la visión artificial, el cual es el efecto de la luz en la percepción de los colores. A parte de eso se reforzará la lógica de control para que el robot sea capaz de seguir la línea de forma más estable y autónoma, al mismo tiempo que es capaz de detectar bifurcaciones y códigos QR, como ya demostró poder hacer en la primera prueba.

6.3. Prueba 3: Conducción Autónoma Completa

Después de los resultados obtenidos en las pruebas anteriores, el trabajo pasa a centrarse en la conducción autónoma completa, puesto que el robot ya es capaz de seguir la línea de forma estable y autónoma.

6.3.1. Descripción de los Componentes Utilizados

Al igual que en la prueba anterior, en esta prueba se ha utilizado el robot con los mismos componentes que en la segunda prueba, con el añadido de una linterna que ayudará a resolver el problema de la iluminación visto en la prueba anterior.

Este nuevo añadido permitirá percibir mejor la línea, puesto que la linterna iluminará la línea y la cámara podrá percibirla mejor. Al mismo tiempo, lo hará menos dependiente de la luz ambiental, por lo que será menos susceptible a fallos en la detección de la línea por cambios en la iluminación.

En la figura 17 se puede observar el robot con la linterna instalada, siendo la versión final de este.



Figura 17: Tercera Versión del Robot

6.3.2. Funcionamiento del Sistema

El funcionamiento del sistema en esta prueba es una combinación de los dos anteriores. El robot deberá seguir la línea de forma autónoma, detectando los colores de la línea y adaptando su velocidad a las curvas. Al mismo tiempo, deberá ser capaz de detectar bifurcaciones y códigos QR, como ya demostró poder hacer en la primera prueba.

Además de esto, se le han atribuido ciertas propiedades a los colores, debido a que el circuito se ha diseñado pensando en estas propiedades. Las propiedades de cada uno de los cuatro colores son:

- **Negro:** El robot deberá aumentar su velocidad al detectar el color negro.

- **Amarillo:** El robot deberá reducir considerablemente su velocidad al detectar el color amarillo, mantendrá la cámara en movimiento horizontal para mejorar el seguimiento de la línea y el tiempo de búsqueda de la línea será superior al de los otros colores, antes de ejecutar la maniobra de marcha atrás.
- **Blanco:** El robot deberá mantener una velocidad algo menor a la velocidad crucero y mantendrá la cámara en movimiento horizontal para mejorar el seguimiento de la línea.
- **Rojo:** El robot se detendrá en seco y moverá la cámara verticalmente, con el objetivo de escanear un código QR, que le indicará que color debe seguir en la próxima bifurcación. Dicha indicación puede ser para un solo ciclo (una vuelta al recorrido hasta llegar de nuevo al color rojo) o puede ser directamente una instrucción que cumpla a lo largo de varios ciclos.

6.3.3. Circuito Desarrollado

En la figura 18 se observa el circuito diseñado para la prueba. Al tratarse en este caso de la prueba final, el circuito ha implementado todas las características que se han abordado en las pruebas anteriores.

En este caso, el circuito se ha diseñado de forma cíclica, como se puede observar en la figura 18, puesto que el robot deberá seguir la línea de forma autónoma, detectando los colores de la línea y adaptándose a ellos.

Como se ha especificado en el apartado anterior, el robot deberá avanzar por el circuito hasta llegar a la línea con color rojo, donde deberá detenerse y escanear un código QR, que le indicará que color o la secuencia que debe seguir a continuación.

Tras escanear el código QR, el robot podrá tomar por el camino derecho (color amarillo) donde se encontrará curvas de 180 grados de color amarillo (debiendo adaptar la velocidad y seguir la línea con la ayuda del movimiento de la cámara) y rectas de color negro (pensadas para que aumente la velocidad). Por otro lado, también podrá tomar por el camino izquierdo (color blanco) donde se encontrará trazado sinuoso con curvas largas y rectas cortas de color blanco (pensada para que mantenga una velocidad algo menor a la velocidad crucero y mantenga la cámara en movimiento horizontal para mejorar el seguimiento de la línea), todo ello seguido de un tramo de una recta de color negro, que lo llevará de nuevo a la línea con color rojo, completando así el ciclo.

De esta forma el recorrido intercala tramos donde obligan al robot a ser preciso para no perder la línea y tramos donde se le permite aumentar la velocidad, poniendo a prueba su capacidad de adaptación. Gracias a ello hará también uso de sus maniobras para encontrar y/o recuperar la línea, puesto que en algunas curvas será fácil que pueda llegar a perderla.



Figura 18: Vista del Tercer Circuito.

6.3.4. Requisitos a Cumplir

Teniendo ya experiencia de la primera y segunda prueba, los objetivos tratan de abarcar todo lo aprendido en las pruebas anteriores. En la tabla 21 se recogen una mezcla de los requisitos de las pruebas anteriores y nuevos requisitos que pretenden evaluar la conducción del robot.

Tabla 21: Requisitos funcionales (RF) de la Tercera Prueba.

Código	Descripción
RF01	El sistema debe ser capaz de detectar líneas de colores amarillo, negro, blanco y rojo.
RF02	El sistema debe ser capaz de seguir las líneas detectadas en el requisito RF-01 en tiempo real e independientemente del color que tengan (siempre y cuando sea alguno de los colores mencionados en el requisito RF-01).
RF03	El sistema debe adaptarse al color que tenga la línea.
RF04	En caso de pérdida de línea, el sistema debe girar en la dirección donde se vio la línea por ultima vez, con el fin de recuperar la línea .
RF05	En caso de que al producirse el requisito RF-04 no se recupere la línea, el sistema debe girar en la dirección opuesta a la que se vio la línea por ultima vez y dar marcha atrás, con el fin de reposicionarse y recuperar la línea .
RF06	El sistema debe ser capaz de detectar la presencia de códigos QR en el entorno con el uso de la cámara.
RF07	El sistema debe ser capaz de detectar cuando se encuentra en una bifurcación (color rojo) y actuar en consecuencia.
RF08	El sistema debe ser capaz de interpretar las instrucciones proporcionadas por el QR leído (RF-06).

6.3.5. Resultados Obtenidos

Tras poner a prueba el robot en el circuito se han sacado las siguientes conclusiones:

- **Seguimiento de línea:** El robot, como era de esperar tras haber realizado la segunda prueba, ha sido capaz de seguir la línea sin gran complicación, adaptandose al color de la línea, cumpliendo por tanto con el requisito **RF-01**, **RF-02** y **RF-03**. En esta ocasión se ha podido observar una mayor precisión a la hora de seguir la línea. Por otro lado, el robot ha perdido la línea en algunas curvas, lo que ha permitido ver su capacidad de recuperación, la cual se ha mejorado al permitir el movimiento de la cámara en ciertos ángulos, cumpliendo de nuevo con el requisito **RF-04** y **RF-05**.
- **Detección de bifurcaciones:** El robot ha podido detectar las bifurcaciones sin ningún problema (color de línea roja) y actuar en consecuencia, según las indicaciones de los códigos QR leídos, pudiendo realizar varios ciclos con una sola instrucción leída, cumpliendo por tanto con los requisitos **RF-06**, **RF-07** y **RF-08**.

6.3.6. Conclusiones de la Prueba 3

Gracias a la sucesión de pruebas por las que ha ido pasando el robot, se han permitido ver los fallos y fortalezas del sistema, permitiendo la corrección de las partes necesarias, haciendo de este modo que la conducción autónoma inicial mejorase. Debido a ello, se ha podido cumplir con el requisito **RF-11** de la Tabla 2. La robustez del sistema a la hora de

detectar y seguir la línea ha sido fruto de una lógica de control, que es capaz de procesar los datos y calcular la trayectoria (error de posición) adecuada para el correcto recorrido del circuito.

6.4. Prueba 4: Prueba del Módulo TrackLine

Como se ha explicado anteriormente, la interfaz del cliente permite el manejo manual del robot, al mismo tiempo, desde dicha interfaz se puede activar el módulo `trackLine` para que el robot entre en el modo autónomo de seguimiento de línea.

6.4.1. Descripción de los Componentes y el Sistema

Para la realización de esta prueba se van a usar los componentes y el sistema implementados en la Prueba 3 (Figura 17).

6.4.2. Circuito Desarrollado

En esta prueba se hará uso del circuito de la Prueba 3 (Figura 18). Ya que es el circuito más complejo.

6.4.3. Requisitos a Cumplir

El objetivo de esta prueba es observar si el robot cumple con los requisitos funcionales **RF-09** y **RF-12** (Tabla 2). Para ello el robot será puesto en marcha en el modo `trackLine` desde la interfaz del cliente. Cuando el robot circule siguiendo la línea con normalidad, se desactivará el módulo `trackLine` desde la interfaz del cliente.

Al tratarse de una prueba basada en el cumplimiento de un requisito funcional del proyecto, se espera que el robot deje de seguir la línea al desactivar el módulo `trackLine`, cumpliendo por tanto con el requisito **RF-09** al poder activar y desactivar el módulo de comportamiento, y cumpliendo el requisito **RF-12** al poder interactuar manualmente con el robot desde la interfaz del cliente. Es por ello también, que esta prueba no contará con requisitos propios

6.4.4. Resultados Obtenidos

La prueba ha comenzado con la activación del módulo `trackLine` desde la interfaz del cliente como se ve en la Figura 19, permitiendo al robot seguir la línea del circuito con normalidad.

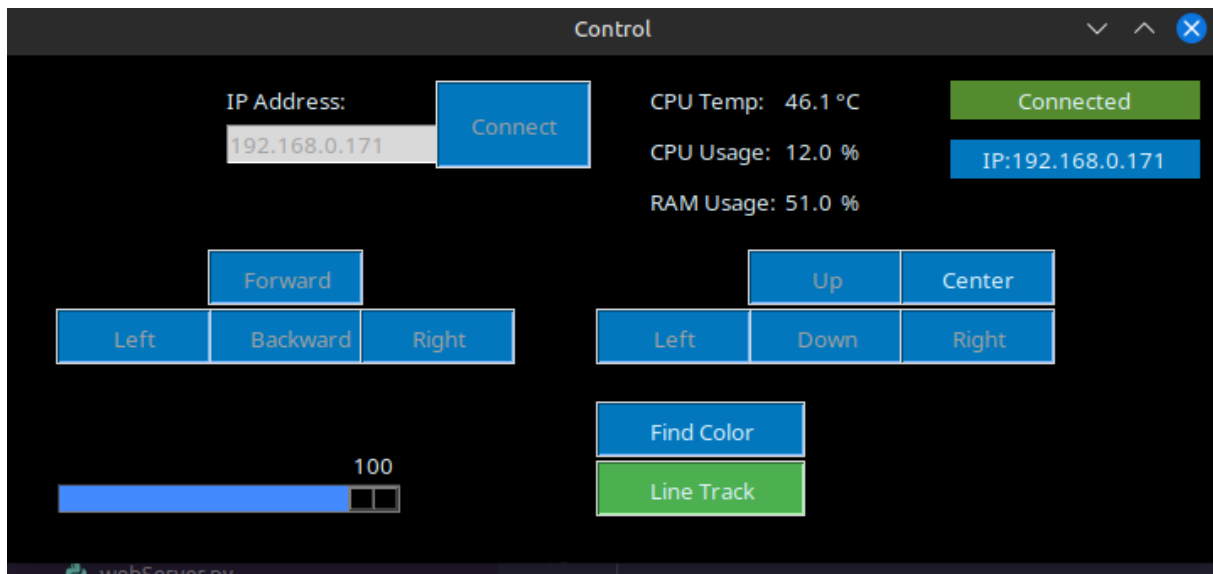


Figura 19: Activación del módulo `trackLine` desde la interfaz del cliente

Se han realizado varios intentos de la prueba. Al desactivar el módulo `trackLine`, como se puede ver en la imagen 20, los resultados han sido los siguientes:

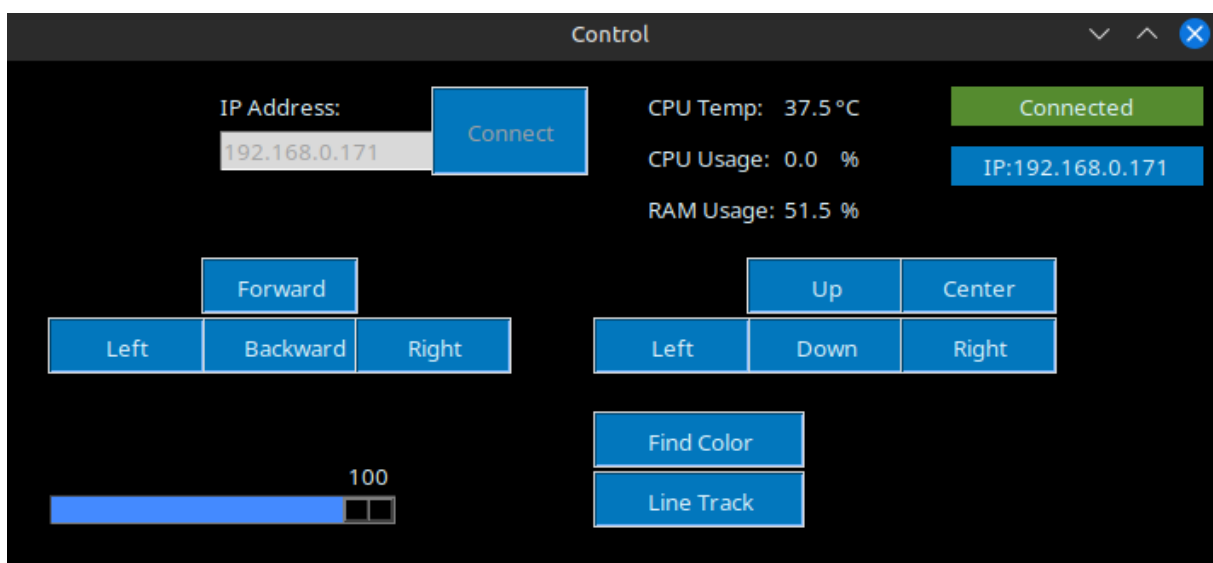


Figura 20: Desactivación del módulo `trackLine` desde la interfaz del cliente

- **Desactivación de TrackLine en línea recta:** Al desactivar el módulo `trackLine` mientras el robot circula en línea recta, el robot deja de detectar y seguir la línea, continua avanzando sin corregir su posición, abandonando eventualmente el circuito. Se ha observado que el robot se puede controlar manualmente desde la interfaz del cliente, interrumpiendo su movimiento continuo de esta forma.
- **Desactivación de TrackLine en curva:** Al desactivar el módulo `trackLine` mientras el robot circula en curva, el robot deja de detectar y seguir la línea, continua girando sin corregir su posición, dando vueltas en círculos de forma indefinida. Se ha observado que el robot se puede controlar manualmente desde la interfaz del cliente, interrumpiendo su movimiento continuo de esta forma.

- **Desactivación de TrackLine en maniobra marcha atrás:** Al desactivar el módulo `trackLine` mientras el robot circula en maniobra marcha atrás, el robot deja de detectar y seguir la línea, continua retrocediendo de forma indefinida aunque encuentre la línea. Se ha observado que el robot se puede controlar manualmente desde la interfaz del cliente, interrumpiendo su movimiento continuo de esta forma.

6.4.5. Conclusiones de la Prueba 4

Una vez obtenidos los resultados de los diferentes intentos de la prueba, se puede concluir que el robot cumple con el requisito funcional **RF-09** ya que es capaz de activar y desactivar el módulo `trackLine` sin ningún problema, pasando al control manual e interrumpiendo la conducción indefinida en la que se encuentra al desactivar el módulo, cumpliendo por tanto a su vez con el requisito **RF-12** al poder interactuar manualmente con el robot desde la interfaz del cliente.

6.5. Prueba 5: Prueba de Búsqueda de línea

A lo largo de las tres primeras pruebas se ha abordado la capacidad del sistema para intentar recuperar la línea en caso de pérdida. Sin embargo, ese no era el punto central de las pruebas, a pesar que el robot ha demostrado ser capaz de recuperar la línea en caso de pérdida, no se ha realizado ninguna prueba para comprobar la capacidad del sistema para encontrar la línea en caso de no tenerla visible desde el inicio. Si bien, la lógica del sistema de búsqueda de línea no se ha desarrollado pensando en esta situación, se ha decidido realizar una prueba para comprobar su comportamiento.

6.5.1. Descripción de los Componentes y el Sistema

Para la realización de esta prueba se van a usar los componentes y el sistema implementados en la Prueba 3 (Figura 17).

6.5.2. Circuito Desarrollado

En esta prueba se hará uso del circuito de la Prueba 3 (Figura 18). A diferencia de la Prueba 3, el robot no comenzará en un punto del circuito, sino que se situará en un punto en el que no haya una línea visible.

En la imagen 21 se puede observar uno de los intentos de la prueba, donde se ve la posición del robot y la posición de la línea más cercana al robot.



Figura 21: Vista del robot buscando la línea

6.5.3. Requisitos a Cumplir

El objetivo de esta prueba es observar si el robot cumple con el requisito funcional **RF-10** (Tabla 2). Para ello el robot será puesto en marcha en el modo **trackLine** en un sitio donde no tenga una línea visible que seguir. Se espera que el robot, siguiendo la lógica de control del sistema y teniendo en cuenta que no tiene una referencia previa de donde puede estar la línea (a diferencia de cuando pierde la línea de vista en mitad del recorrido del circuito, que sabe la dirección por donde ha perdido la línea), actúe de la forma esperada hasta que encuentre la línea en caso de que esta se encuentre dentro del campo de visión de la cámara en algún momento.

La forma de actuar esperada según la lógica de control es la siguiente:

1. El robot inicia el modo **trackLine** y, al no detectar ninguna línea, entra en estado de búsqueda.
2. El sistema empieza a hacer varios barridos con la cámara horizontalmente.
3. En caso de no encontrar la línea en ninguno de los barridos, el robot retrocede una pequeña distancia y repite el proceso.
4. En el momento en que la línea entra en el área de detección, el controlador calcula el error de desplazamiento.
5. El robot corrige su trayectoria para centrar la línea y procede con la navegación normal.

6.5.4. Resultados Obtenidos

Una vez finalizada la pruebas, los resultados obtenidos han sido los esperados. El robot ha seguido la secuencia de barrido y retroceso mencionada en el punto anterior hasta encontrar la línea y modificar su trayectoria para acercarse y continuar el recorrido. Se han hecho varios intentos a distintas distancias y los resultados han sido los mismos en los casos que la línea ha entrado en su rango de visión. En los casos donde la línea no ha entrado en su campo de visión, el robot ha continuado realizando la secuencia de barrido de cámara y retroceso indefinidamente.

6.5.5. Conclusiones

Tras los resultados obtenidos se puede calificar el requisito **RF-10** como cumplido, debido a que las maniobras implementadas en la pérdida de la línea han sido ejecutadas de acuerdo a lo establecido. De esta forma se le ha dotado al robot de un sistema de prevención en el caso de la pérdida o falta de línea visible.

Capítulo 7

Conclusiones

Tras finalizar el diseño, implementación y validación experimental del sistema, se presentan las conclusiones obtenidas, evaluando el grado de cumplimiento de los objetivos iniciales y analizando el impacto de las decisiones técnicas adoptadas.

7.1. Conclusiones Generales

El presente Trabajo Fin de Grado ha logrado desarrollar con éxito un prototipo de vehículo autónomo capaz de realizar una navegación contextual en tiempo real sobre una plataforma de hardware limitado (Raspberry Pi 3 B+).

La principal aportación del proyecto ha sido demostrar que es posible superar las limitaciones de la robótica reactiva tradicional (basada en sensores infrarrojos) mediante una arquitectura de software eficiente. La implementación de un sistema de visión artificial multiproceso ha permitido dotar al robot de una capacidad de interpretación semántica (detección de colores y códigos QR) sin comprometer la fluidez del control de los motores, validando la hipótesis de que la visión artificial es viable en sistemas embebidos de bajo coste si se optimiza el uso de la CPU.

7.2. Grado de Cumplimiento de los Objetivos

El objetivo principal de este Trabajo de Fin de Grado era el diseño e implementación de una arquitectura modular para la navegación autónoma contextual en sistemas de bajo coste. Para evaluar el éxito del proyecto de manera objetiva, se procede a verificar el cumplimiento de los requisitos formales establecidos en el capítulo de Análisis.

A continuación, se presenta la trazabilidad entre los requisitos planteados y la solución final implementada.

7.2.1. Validación de Requisitos Funcionales

La Tabla 22 detalla el grado de cobertura de las funciones operativas del vehículo.

Tabla 22: Grado de cumplimiento de los Requisitos Funcionales (RF).

Cód.	Descripción Resumida	Estado	Validación / Observaciones
RF01	Control independiente de motores (tracción y giro).	Cumplido	Módulos <code>move.py</code> y <code>RPIservo.py</code> operativos.
RF02	Capa de Abstracción de Hardware (HAL).	Cumplido	Implementada de módulos como <code>move.py</code> , <code>RPIservo.py</code> , <code>camera_opencv.py</code> y <code>robotLight.py</code> que actúan como drivers de los componentes físicos.
RF03	Detección de línea en tiempo real.	Cumplido	Algoritmo de visión permite un tiempo de respuesta suficientemente rápido para reaccionar y adaptarse al entorno.
RF04	Detección de contexto (marcas de color).	Cumplido	Discriminación correcta de líneas de distintos colores.
RF05	Detección de señales/códigos.	Cumplido	Implementada detección de códigos QR con la librería <code>pyzbar</code> .
RF06	Control PID de trayectoria.	Cumplido	El vehículo corrige oscilaciones y mantiene el centro del carril.
RF07	Gestor de eventos contextuales.	Cumplido	La Máquina de Estados gestiona transiciones (Stop, Espera, Avance).
RF08	Comprensión de señales y/o códigos detectados.	Cumplido	El vehículo se detiene ante un evento y comprende los códigos percibidos.
RF09	Activación y desactivación de comportamientos.	Cumplido	El vehículo activa y desactiva comportamientos según las órdenes del cliente.
RF10	Protocolo de pérdida de referencia.	Cumplido	El robot busca la línea por el lado perdido o ejecuta otras maniobras para recuperar la referencia.
RF11	Cálculo de desviación lateral.	Cumplido	Cálculo preciso del error C_x en píxeles.
RF12	Interfaz de usuario.	Cumplido	Web App desarrollada con Flask para control y telemetría.

7.2.2. Validación de Requisitos No Funcionales

Del mismo modo, se analiza el cumplimiento de las restricciones de diseño y calidad en la Tabla 23.

Tabla 23: Grado de cumplimiento de los Requisitos No Funcionales (RNF).

Cód.	Restricción	Estado	Justificación
RNF01	Autonomía energética.	Cumplido	El prototipo cuenta con baterías 18650.
RNF02	Reusabilidad y mantenibilidad.	Cumplido	Código modular y documentado en el manual de código.
RNF03	Hardware COTS (Comercial).	Cumplido	Uso exclusivo de Raspberry Pi y componentes genéricos.
RNF04	Licencia Open Source.	Cumplido	Repositorio en GitHub con todo el código fuente ⁹ .
RNF05	Baja latencia (Tiempo Real).	Cumplido	Procesamiento optimizado con el uso de la librería multiprocessing y la distribución del trabajo en varios hilos.
RNF06	Documentación de replicado físico.	Cumplido	Esquemas de conexión incluidos en la memoria y en el manual de usuario.
RNF07	Documentación de despliegue del sistema.	Cumplido	Manual de usuario con toda la documentación para replicar el sistema y ponerlo en marcha.
RNF08	Tolerancia a iluminación.	Parcial	El sistema funciona en interiores, pero requiere calibración de umbrales HSV si la luz cambia drásticamente.
RNF09	Bajo Coste.	Cumplido	Los componentes han sido elegidos con un presupuesto limitado y teniendo en cuenta el bajo coste.

Los requisitos funcionales y no funcionales que se han abordado en estas tablas provienen de las tablas 2 y 3 respectivamente.

7.2.3. Conclusión sobre los Objetivos

A la vista de los resultados, se puede afirmar que el objetivo principal se ha alcanzado satisfactoriamente. Se ha logrado transformar una plataforma robótica básica en un sistema autónomo capaz de interpretar su entorno, superando las limitaciones de los seguidores de línea reactivos tradicionales.

Aunque el requisito **RNF08** (tolerancia lumínica) se ha cumplido parcialmente, una limitación inherente a los algoritmos de visión clásica basados en color, la arquitectura propuesta permite la futura integración de técnicas de *Deep Learning* para solventar este problema sin necesidad de rediseñar el sistema completo, validando así el enfoque modular del proyecto.

7.3. Retos Técnicos y Limitaciones

Durante el desarrollo surgieron desafíos significativos que moldearon la solución final:

- **La limitación del GIL en Python:** Inicialmente, el uso de *threads* provocaba cuellos de botella donde el procesamiento de imagen bloqueaba el PWM de los motores. Esto forzó la migración hacia una arquitectura de multiprocesamiento (*multiprocessing*), aumentando la complejidad de la comunicación entre procesos pero garantizando el tiempo real.
- **Sensibilidad a la Iluminación:** Como se ha mencionado anteriormente en el apartado de conclusiones sobre los objetivos el problema de la iluminación no es algo que se haya solventado en todos los casos. Aunque el espacio de color HSV mejora la robustez frente al RGB, el sistema aún presenta sensibilidad ante cambios drásticos de luz solar directa. Esto requirió una calibración fina de los umbrales de los perfiles de color (*VISION_PROFILE*) y el añadido de una linterna con el fin de estabilizar los rangos HSV que captaba la cámara, de forma que no fuera tan susceptible al cambio de iluminación del entorno.
- **Sincronización de servomotores y cámara:** Al implementar el seguimiento de línea por visión artificial, el sistema calcula la distancia del centro de la cámara a la línea, para hallar el error y mover el servomotor que controla las ruedas (*steering*) en busca de la línea. El problema reside en que el sistema trata constantemente de situar la línea en el centro de la imagen, lo que se acaba traduciendo en una oscilación continua por parte del coche sobre la línea, que empeora cuanto más aumente la velocidad. Lograr una navegación más fluida exigió un ajuste extremadamente fino de las constantes del controlador PID (especialmente la componente derivativa K_d).

7.4. Líneas de Trabajo Futuro

El diseño modular del proyecto permite plantear futuras ampliaciones que exceden el alcance de este TFG pero que aportarían gran valor:

1. **Migración a Deep Learning:** Sustituir el procesamiento de imagen clásico (filtros HSV) por una Red Neuronal Convolutiva (CNN) ligera, como Tiny-YOLO [31], ejecutada en un acelerador hardware (ej. Google Coral USB). Esto permitiría detectar objetos complejos (señales de tráfico reales, peatones) en lugar de solo colores y códigos QR, además de mejorar la robustez ante cambios de iluminación.
2. **Fusión de Sensores (IMU):** Integrar un acelerómetro y giroscopio para combinar la odometría visual con la inercial. Esto permitiría al robot mantener la trayectoria durante breves instantes si la cámara pierde la línea por un deslumbramiento o fallo técnico.

3. **Navegación SLAM:** Implementar algoritmos de localización y mapeo simultáneos para que el robot no solo siga una línea, sino que construya un mapa de su entorno y pueda planificar rutas óptimas en un espacio abierto [25].
4. **Implementación de sensores de distancia:** Añadir sensores de distancia para detectar obstáculos y evitar colisiones, de forma que el robot pueda reaccionar ante imprevistos y aumentar de esta forma su adaptabilidad al entorno, optando por no permanecer simplemente encima de la línea, si no que tendría en cuenta el entorno que le rodea.

Bibliografía

- [1] «Embedded Software,» en *Advances in Computers*, vol. 56, Elsevier, 1 de ene. de 2002, págs. 55-95. DOI: [10.1016/S0065-2458\(02\)80004-3](https://doi.org/10.1016/S0065-2458(02)80004-3). dirección: <https://www.sciencedirect.com/science/chapter/bookseries/abs/pii/S0065245802800043> (visitado 19-01-2026).
- [2] alpoma. «Elmer y Elsie, las tortugas robot de 1948,» Tecnología Obsoleta. (17 de mayo de 2015), dirección: <https://alpoma.net/tecob/?p=11359> (visitado 12-01-2026).
- [3] inserrobotica. «Mobile Industrial Robots: la revolución en el entorno industrial,» Inser Robótica S.A. (31 de mar. de 2021), dirección: <https://inser-robotica.com/mobile-industrial-robots-revolucion-entorno-industrial/> (visitado 12-01-2026).
- [4] «Stanford's robotics legacy.» (), dirección: <https://news.stanford.edu/stories/2019/01/stanfords-robotics-legacy> (visitado 12-01-2026).
- [5] H. P. Moravec, «The Stanford Cart and the CMU Rover,» en *Autonomous Robot Vehicles*, I. J. Cox y G. T. Wilfong, eds., New York, NY: Springer, 1990, págs. 407-419, ISBN: 978-1-4613-8997-2. DOI: [10.1007/978-1-4613-8997-2_30](https://doi.org/10.1007/978-1-4613-8997-2_30). dirección: https://doi.org/10.1007/978-1-4613-8997-2_30 (visitado 12-01-2026).
- [6] *Vehículo de Braitenberg*, en *Wikipedia, la enciclopedia libre*, 14 de feb. de 2025. dirección: https://es.wikipedia.org/w/index.php?title=Veh%C3%ADculo_de_Braitenberg&oldid=165414309 (visitado 12-01-2026).
- [7] «¿Que es Raspberry Pi? - Raspberry Pi.» (14 de feb. de 2019), dirección: <https://raspberrypi.cl/que-es-raspberry/> (visitado 12-01-2026).
- [8] *OpenCV*, en *Wikipedia*, 18 de dic. de 2025. dirección: <https://en.wikipedia.org/w/index.php?title=OpenCV&oldid=1328268220> (visitado 12-01-2026).
- [9] C. Márquez-Sánchez, M. Antonio-Cruz, J. Sandoval-Gutiérrez, V. E. Quiroz-Velázquez y C. A. Merlo-Zapata, «Construction of a Low-cost Wheeled Mobile Robot for Testing Automatic Control Techniques,» en *2019 International Conference on Mechatronics, Electronics and Automotive Engineering (ICMEAE)*, nov. de 2019, págs. 102-107. DOI: [10.1109/ICMEAE.2019.00026](https://doi.org/10.1109/ICMEAE.2019.00026). dirección: <https://ieeexplore.ieee.org/abstract/document/9140169> (visitado 19-01-2026).
- [10] J. Khalfallah y J. Ben Hadj Slama, «The effect of emotional analysis on the improvement of experimental e-learning systems,» *Computer Applications in Engineering Education*, vol. 27, n.º 2, págs. 303-318, 2019, ISSN: 1099-0542. DOI: [10.1002/cae.22075](https://doi.org/10.1002/cae.22075). dirección: <https://onlinelibrary.wiley.com/doi/abs/10.1002/cae.22075> (visitado 19-01-2026).
- [11] I. Terzic, I. Boticki, M. Lovrekovic y L. Topalovic, «Educational Robotics through Web Applications: From Visual Programming to Simulation-Driven Learning,» *In-*

- ternational Conference on Computers in Education*, 1 de dic. de 2025, ISSN: 3078-4360. dirección: <https://library.apsce.net/index.php/ICCE/article/view/5673> (visitado 19-01-2026).
- [12] J. M. Anderson, N. Kalra, K. D. Stanley, P. Sorensen, C. Samaras y T. A. Oluwatola, «Autonomous Vehicle Technology: A Guide for Policymakers,» 22 de mar. de 2016. dirección: https://www.rand.org/pubs/research_reports/RR443-2.html (visitado 19-01-2026).
 - [13] «Safety,» Waymo. (), dirección: <https://waymo.com/safety/> (visitado 19-01-2026).
 - [14] «Full Self-Driving (Supervised),» Tesla. (), dirección: <https://www.tesla.com/fsd> (visitado 19-01-2026).
 - [15] M.-B. Group. «First internationally valid system approval: Conditionally automated driving,» Mercedes-Benz Group. (9 de dic. de 2021), dirección: <https://group.mercedes-benz.com/innovation/product-innovation/autonomous-driving/system-approval-for-conditionally-automated-driving.html> (visitado 25-09-2025).
 - [16] K. Ozbay, X. (Ban y C. Y. D. Yang, «Developments in Connected and Automated Vehicles,» *Journal of Intelligent Transportation Systems*, vol. 22, n.º 3, págs. 187-189, 4 de mayo de 2018, ISSN: 1547-2450. DOI: [10.1080/15472450.2018.1466407](https://doi.org/10.1080/15472450.2018.1466407). dirección: <https://doi.org/10.1080/15472450.2018.1466407> (visitado 19-01-2026).
 - [17] R. Ayala y T. K. Mohd, «Sensors in Autonomous Vehicles: A Survey,» *Journal of Autonomous Vehicles and Systems*, vol. 1, n.º 031003, 27 de dic. de 2021, ISSN: 2690-702X. DOI: [10.1115/1.4052991](https://doi.org/10.1115/1.4052991). dirección: <https://doi.org/10.1115/1.4052991> (visitado 19-01-2026).
 - [18] T. Litman, «Implications for Transport Planning,»
 - [19] «Line following robot for library inventory management system,» ResearchGate. (), dirección: https://www.researchgate.net/publication/241179898_Line_following_robot_for_library_inventory_management_system (visitado 19-01-2026).
 - [20] E. Palmares-Trejo, F. Sánchez-Niño, F. J. de Anda-Salazar y C. Soubervielle-Montalvo, «Robótica educativa: diseño y construcción de un robot seguidor de línea,» *Revista Conexión de Ingeniería*, vol. 5, n.º 9, págs. 8-17, 2021, ISSN: 2683-1872.
 - [21] E. Vázquez-Bonilla, A. Franco-Árcega, V. López-Morales y M. A. Ojeda-Misses, «Experimental Study of Ludoeducational Robotics to Teaching of a Second Language: Human–Robot Interaction and Play Among Students,» *IEEE Revista Iberoamericana de Tecnologías del Aprendizaje*, vol. 17, n.º 4, págs. 358-371, 2022.
 - [22] M. Sanders, «STEM, STEM education, STEMmania,» *The Technology Teacher*, vol. 68, n.º 4, págs. 20-26, 2009.
 - [23] M. I. Abu El-Sebah, F. A. Syam, E. A. Sweelem, M. M. El-Sotouhy y M. M. El Sotouhy, «A Proposed Controller for an Autonomous Vehicles Embedded System,» *WSEAS TRANSACTIONS ON CIRCUITS AND SYSTEMS*, vol. 22, págs. 1-9, 7 de feb. de 2023, ISSN: 2224-266X, 1109-2734. DOI: [10.37394/23201.2023.22.1](https://doi.org/10.37394/23201.2023.22.1). dirección: [https://wseas.com/journals/cas/2023/a025101-001\(2023\).pdf](https://wseas.com/journals/cas/2023/a025101-001(2023).pdf) (visitado 25-02-2025).
 - [24] «ADEEPT PICAR-B USER MANUAL Pdf Download,» ManualsLib. (), dirección: <https://www.manualslib.com/manual/1647846/Adeept-Picar-B.html> (visitado 19-01-2026).
 - [25] C. Cadena, L. Carlone, H. Carrillo et al., «Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age,» *IEEE Transactions*

- on Robotics*, vol. 32, n.º 6, págs. 1309-1332, dic. de 2016, ISSN: 1941-0468. DOI: [10.1109/TR0.2016.2624754](https://doi.org/10.1109/TR0.2016.2624754). dirección: <https://ieeexplore.ieee.org/abstract/document/7747236> (visitado 19-01-2026).
- [26] Raspberry Pi Ltd, *Raspberry Pi 3 Model B+*, Accedido el 29 de enero de 2026, 2024. dirección: <https://www.raspberrypi.com/products/raspberry-pi-3-model-b-plus/>.
- [27] Wikipedia, *Geometría de Ackermann*, Accedido el 29 de enero de 2026, 2026. dirección: https://es.wikipedia.org/wiki/Geometr%C3%ADa_de_Ackermann.
- [28] Raspberry Pi Ltd, *Raspberry Pi Camera Module v2*, Accedido el 29 de enero de 2026, 2024. dirección: <https://www.raspberrypi.com/products/camera-module-v2/>.
- [29] F. López Castro, *Repositorio de código fuente: Trabajo Fin de Grado*, Accedido: 07-02-2026, 2026. dirección: <https://github.com/Felipelc003/Trabajo-Fin-de-Grado>.
- [30] HetPRO Store, *Protocolo de comunicación I2C*, Accedido el 31 de enero de 2026, 2026. dirección: <https://hetpro-store.com/TUTORIALES/i2c/>.
- [31] Y. Zhang, M. Ye, G. Zhu, Y. Liu, P. Guo y J. Yan, «FFCA-YOLO for Small Object Detection in Remote Sensing Images,» *IEEE Transactions on Geoscience and Remote Sensing*, vol. 62, págs. 1-15, 2024, ISSN: 1558-0644. DOI: [10.1109/TGRS.2024.3363057](https://doi.org/10.1109/TGRS.2024.3363057). dirección: <https://ieeexplore.ieee.org/abstract/document/10423050> (visitado 19-01-2026).

Apéndice A

Manual de Usuario

En esta parte del documento se van a detallar los pasos que tiene que seguir el usuario para poner en marcha el sistema.

A.1. Requisitos Previos

Para la puesta en marcha del sistema se requiere:

- Kit robótico Adeept PiCar-B.
- Raspberry Pi 3 B+ o superior.
- 2 baterías 18650 recargables.
- Tarjeta MicroSD (mínimo 16GB) con Raspberry Pi OS.

A.2. Montaje e instalación del robot

Para el montaje e instalación del robot se recomienda seguir el [Manual-PiCar-B](#) en caso de haber adquirido el kit *Adeapt PiCar-B Mars Rover*.

Durante el montaje del robot no será necesario seguir el manual al pie de la letra, sino que se puede seguir de forma orientativa, ya que se han realizado modificaciones en el montaje original del robot, se pueden prescindir de algunos componentes como lo son el sensor *Adeapt Ultrasonic*, los 4 leds *CH WS2812 RGB LED* o el sensor de infrarrojos *CH Line Tracking Module* ya que no se utilizan en el sistema. Por otro lado, se ha añadido una linterna en la parte frontal del robot, que no viene incluida en el kit, para mejorar la calidad de la imagen captada por la cámara.

En la figura 22 se muestra como debe quedar el montaje del robot:

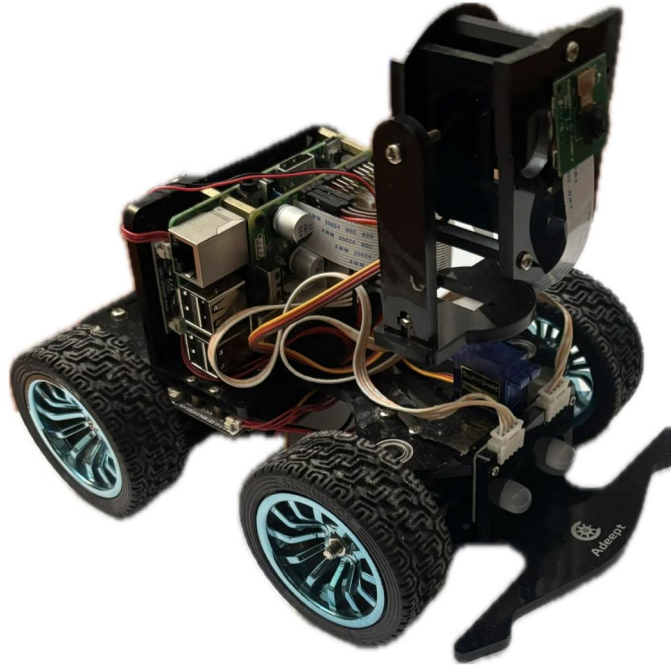


Figura 22: Vista final del robot ensamblado.

A.3. Instalación del Entorno Software

Tras el montaje del robot, queda la instalación del sistema operativo. Para ello se instalará el programa *Raspberry Pi Imager* en un ordenador, el cual se puede descargar desde la página oficial de Raspberry Pi. Este programa permite instalar el sistema operativo en una tarjeta micro SD, la cual se insertará en la Raspberry Pi (Se recomienda usar la versión más actualizada del sistema operativo, *Raspberry Pi OS (Bookworm)*). Una vez instalado el sistema operativo, se deberá configurar la conexión a la red WiFi a usar, conectando la placa a un monitor y un teclado, si no se hizo anteriormente.

A.3.1. 1. Preparación del Sistema Operativo

Tras la instalación del Sistema Operativo hay que seguir el manual de instalación del robot, usando el programa PuTTY en caso de usar Windows o la terminal en caso de usar Linux, para acceder al robot por ssh de forma remota.

Una vez se pueda acceder al robot de forma remota, será necesario habilitar el bus I2C y la cámara como se muestra en manual de instalación. Estos son los comandos que se deben usar para ello:

```
sudo apt-get update
sudo apt-get upgrade
sudo raspi-config
# Activar: Interface Options -> Camera / I2C / SSH
```

El siguiente paso será clonar el repositorio del proyecto en el robot usando el comando:

```
git clone https://github.com/Felipelc003/Trabajo-Fin-de-Grado.git
```

Habiendo ya clonado el repositorio, se deberá instalar las dependencias necesarias para el correcto funcionamiento del proyecto. Para ello se ejecutará el comando:

```
sudo chmod +x install.sh
sudo ./install.sh
```

A.4. Ejecución del servidor y el cliente

Tras lo anterior, solo quedará ejecutar el servidor con el comando:

```
cd server/
python3 webServer.py
```

El siguiente paso será clonar el repositorio en el ordenador que se vaya a usar como cliente. Una vez lo tengamos clonado deberemos tener python instalado y ejecutar el siguiente comando en una terminal para instalar las librerías necesarias:

```
pip install opencv-python numpy websockets
```

Tras ello habrá que usar el entorno virtual para ejecutar el cliente `client/GUI.py` con los comandos:

```
source venv/bin/activate # Activar el entorno virtual
cd client/
python3 GUI.py # Ejecutar el cliente
```

Tras la ejecución del cliente nos debe salir una ventana como la que se observa en la figura 23:

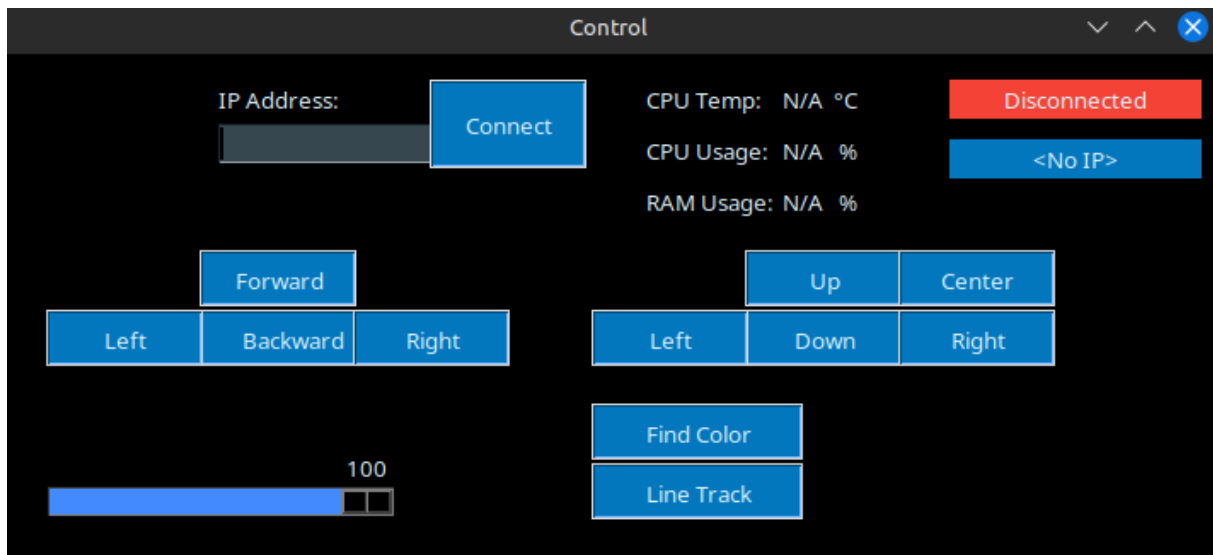


Figura 23: Interfaz del cliente

Solo quedaría introducir la dirección IP del servidor en el campo correspondiente y pulsar el botón de conectar. Para saber qué dirección IP hay que introducir podemos mirarlo en la última línea de la terminal donde se está ejecutando el servidor:

```
[RPIServo] Controlador PCA9685 iniciado correctamente.
[Init] Iniciando hardware...
LEDs delanteros inicializados.
[Move] Error iniciando PWM: A PWM object already exists for this GPIO channel
[0:30:16.414173140] [2099] INFO Camera camera_manager.cpp:330 libcamera v0
.5.2+99-bfd68f78
[0:30:16.475244469] [2112] INFO IPAProxy ipa_proxy.cpp:180 Using tuning file /
usr/share/libcamera/ipa/rpi/vc4/ov5647.json
[0:30:16.492745384] [2112] INFO Camera camera_manager.cpp:220 Adding camera '/
base/soc/i2c0mux/i2c@1/ov5647@36' for pipeline handler rpi/vc4
[0:30:16.493200852] [2112] INFO RPI vc4.cpp:440 Registered camera /base/soc/
i2c0mux/i2c@1/ov5647@36 to Unicam device /dev/media0 and ISP device /dev/
media1
[0:30:16.510319164] [2099] INFO Camera camera.cpp:1215 configuring streams: (0)
640x480-XBGR8888/sRGB (1) 640x480-SGBRG10_CSI2P/RAW
[0:30:16.511334006] [2112] INFO RPI vc4.cpp:615 Sensor: /base/soc/i2c0mux/i2c@1
/ov5647@36 - Selected sensor format: 640x480-SGBRG10_1X10/RAW - Selected
unicam format: 640x480-pGAA/RAW
[BaseCamera] Hilo de cámara iniciado.
[Init] Cámara iniciada.
[Init] Servidor Flask en puerto 5000
[Network] Wi-Fi conectado.
[Init] Servidor WebSocket en puerto 8888... Listo.
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
```

```
* Running on http://192.168.0.171:5000 # <-- Esta es la direccion IP que hay  
que introducir en GUI.py
```

Tras conectar con el servidor, se abrirá otra ventana que nos permitirá ver lo que el robot ve a través de la cámara.

Con la interfaz de control (imagen 23) podremos controlar el robot de forma remota, haciendo uso de los botones que se muestran en la imagen o de las teclas WASD del teclado. Finalmente para que el robot siga la línea solo habrá que pulsar el botón de "TrackLine".